Gold Prices Project

PGP DSBA – Guided Project Report

Aravapalli.Naga Surendera Babu
2-16-2025

Contents

1. Description: - Gold Prices Data Analysis: - 6

2. Data Description 6

3. Overview of the Dataset 6

3.1 Displaying few rows of the dataset: - 6

3.2 Displaying few rows of the dataset: - 7

3.3 Checking the shape of the dataset: - 7

3.4 Checking the data types of the columns for the dataset 7

3.5 Statistical summary of the dataset 7

3.6 Checking for null values in the data 8

3.7 Checking for missing values in the data 9

4. Exploratory Data Analysis 9

4.1 To find the trend of the data 9

4.2 Resample the data 10

4.3 To find the statistics of the data 10

4.4 To find the trend of price at year level 11

4.5 To plot a boxplot at year level 11

4.6 To plot a boxplot at month level 12

4.7 To create a pivot table to display monthly price across years 12

4.8 To plot trend of months across years 13

4.9 Decomposition 14

4.9.1 Decompose the time series additively 14

4.9.2 Decompose the time series multiplicative 15

5. **Data Pre-processing** 16

5.1 Split the data into train and test and plot the training and test data 16

5.2 To check the shape of train and test sets 17

5.3 To check the shape of train and test sets to find the trend of train data 17

6. **Model Building - Original Data** 19

6.1 Linear Regression 19

6.1.1 Now that our training and test data has been modified, let us go ahead use $LinearRegression$ to build the model on the training data and test the model on the test data. 20

6.1.2 Test Data - RMSE 20

6.2 Moving Average (MA) 20

6.2.1 Trailing Moving Average 21

6.2.2 Plotting on the whole data 22

6.2.3 Let us split the data into train and test and plot this Time Series. The window of the moving average is need to be carefully selected as too big a window will result in not having any test set as the whole series might get averaged over. 22

6.2.4 Before we go on to build the various Exponential Smoothing models, let us plot all the best models and compare the Time Series plots 23

6.3 Simple Exponential Smoothening Models 23

6.3.1 Define the simple exponential smoothening model 23

6.3.2 Forecast the model 24

6.3.3 Plotting on both the Training and Test data 24

6.3.4 Model Evaluation for α = 0.995 : Simple Exponential Smoothing 25

6.4 Double Exponential Smoothening (Holt's Model) 25

6.4.1 fit the model 26

6.4.2 Plotting on both the Training and Test data 26

6.5 Triple Exponential Smoothing (Holt - Winter's Model) 28

6.5.1 Define the triple exponential smoothening model 28

6.5.2 Prediction on the test data 28

6.5.3 Plotting on both the Training and Test using autofit 28

6.5.4 Test Data 29

6.5.5 Fit the model 30

6.5.6 list (TES_test.columns): - 30

6.5.7 Plotting on both the Training and Test data using brute force alpha, beta and gamma determination 47

6.5.7 Sorted by RMSE values on the Test Data: 48

6.5.8 Test for stationarity of the series - Dicky Fuller test 48

7. Model Building - Stationary Data 51

7.1 Plot the Autocorrelation and the Partial Autocorrelation function plots on the whole data. 51

7.2 Auto ARIMA Model 51

7.2.1 Creating an empty Dataframe with column names only 51

7.2.2 Fit the ARIMA model 51

7.2.3 Sort the above AIC values in the ascending order to get the parameters for the minimum AIC value 52

7.2.4 Sort the above AIC values in the ascending order to get the parameters for the minimum AIC value 52

7.2.5 Predict on the Test Set using this model and evaluate the model. 53

7.2.6 Plotting on both the Training and Test using autofit 54

7.2.7 Find the RMSE on test data 55

7.3 ARIMA Model 55

7.3.1 Define the value to make the data stationary 55
7.3.2 Fit the ARIMA model 56
7.3.3 Sort the above AIC values in the ascending order to get the parameters for the minimum AIC value 56
7.3.4 Build the ARIMA model based on the best params 57
7.3.5 Forecast the ARIMA model 57
7.3.6 the RMSE on test data 58
7.3.7 Plotting on both the Training and Test using autofit 58
7.4 Compare the Performance of the model & Forecast 60
7.4.1 Build the best model after comparing all the RMSE values 60
7.4.2 to build the best model to forecast 61
8. **Actionable Insights and Recommendations** 62
8.1 Insights 62
8.2 Recommendations 62

List of Tables: -

Table 1 Top 5 Rows of the data set 6
Table 2 Last 5 Rows of the data set 7
Table 3 Type of Coulmns for the dataset 7
Table 4 Statistical summary 8
Table 5 Null values in the dataset 8
Table 6 Missing values in the data 9
Table 7 Resample the data 10
Table 8 Statistical summary of the data 10
Table 9 Pivot table for monthly prices across years 12
Table 10 First few rows of training data 16
Table 11 First few rows of test data 17
Table 12 First few rows of training data 19
Table 13 First few rows of test data 19
Table 14 Linear Regression model 20
Table 15 LR - Test RMSE 20
Table 16 Moving Average 21
Table 17 Moving Average Trailing 21
Table 18 LR - Test RMSE 23
Table 19 Fit the simple exponential smoothening model 24
Table 20 Forecast the model 24
Table 21 Simple Exponential Smoothing Model forecast on the Test Data 25
Table 22 empty dataframe to store our values 26
Table 23 Forecast the model 26
Table 24 predictions made by the linear regression model 27
Table 25 triple exponential smoothening model 28
Table 26 Forecast the triple exponential smoothening model 28
Table 27 RMSE on test data 29
Table 28 empty dataframe 30
Table 29 Fit the model 30
Table 30 Training and Test data using brute force alpha, beta and gamma determination 48
Table 31 RMSE values on the Test Data 48
Table 32 stationarity of the series - Dicky Fuller test 49
Table 33 perform differencing on train data 50
Table 34 Auto ARIMA Model 51
Table 35 empty Dataframe 51
Table 36 ARIMA model AIC Values 51
Table 37 parameters for the minimum AIC value 52
Table 38 parameters for the minimum AIC value 53
Table 39 data stationary 55
Table 40 ARIMA model 56
Table 41 parameters for the minimum AIC value 56
Table 42 ARIMA model based on the best params 57
Table 43 ARIMA model 59
Table 44 Test using autofit 59
Table 45 Performance of the model & Forecast 60

List of Figures: -

Figure 1 trend of the data 9
Figure 2 trend of price at year level 11
Figure 3 boxplot at year level 11
Figure 4 boxplot at month level 12
Figure 5 trend of months across years 13
Figure 6 Decompose the time series additively 14
Figure 7 Decompose the time series multiplicative 15
Figure 8 shape/trend of train and test sets 18
Figure 9 predictions made by the model 20
Figure 10 Plotting on the whole data 22
Figure 11 plot all the best models and compare the Time Series plots 23
Figure 12 Plotting on both the Training and Test data 25
Figure 13 Plotting on both the Training and Test data 26
Figure 14 Plotting on both the Training and Test using autofit 29
Figure 15 Plotting on both the Training and Test data using brute force alpha, beta and gamma determination 47
Figure 16 stationarity of the series - Dicky Fuller test 49
Figure 17 differencing on train data 50
Figure 18 Plotting on both the Training and Test using autofit 55
Figure 19 Plotting on both the Training and Test using autofit 59
Figure 20 Build the best model after comparing all the RMSE values 61
Figure 21 best model to forecasts 61

1. Description:- Gold Prices Data Analysis:-

Context

In the dynamic landscape of the financial markets, the demand for accurate predictions of gold prices is crucial for investors, traders, and stakeholders. Similar to the challenges faced by business communities in the United States in identifying and attracting the right talent, predicting the future values of precious metals poses a significant challenge. The inherent volatility and complex dynamics influencing gold prices require a reliable forecasting tool.

The goal is to provide a valuable asset for individuals navigating the financial markets, ensuring they can approach the unpredictability of gold prices with confidence and strategic insight.

Objective:

In the realm of financial markets, predicting the future value of gold and silver holds significant importance for investors, traders, and stakeholders. The challenge lies in the inherent volatility and complex dynamics influencing gold prices. To address this, the objective is to develop an accurate time series forecasting model that can predict the future prices of gold. The model should leverage historical gold and silver price data. The anticipated outcome is a robust forecasting tool that empowers stakeholders to make informed decisions and navigate the dynamic landscape of the gold market with confidence.
s

2. Data Description

The data contains the different attributes of leads and their interaction details with the demand for accurate predictions of gold prices. The detailed data dictionary is given below.

3. Overview of the Dataset

3.1 Displaying few rows of the dataset:-

- The data frame has 2 columns.
- Data in each row corresponds to the leads and their interaction details with inherent volatility and complex dynamics influencing gold prices.

	Date	Price
0	08/17/2023	1915.2
1	08/16/2023	1928.3
2	08/15/2023	1935.2
3	08/14/2023	1944.0
4	08/11/2023	NaN

Table 1 Top 5 Rows of the data set

3.2 Displaying few rows of the dataset:-

- The data frame has 2 columns and last 5 rows of the dataset.

	Date	Price
2534	08/23/2013	NaN
2535	08/22/2013	1370.8
2536	08/21/2013	1370.1
2537	08/20/2013	1372.6
2538	08/19/2013	1365.7

Table 2 Last 5 Rows of the data set

3.3 Checking the shape of the dataset:-

- The dataset contains information (2 attributes) about 2539 rows.
  $(2539, 2)$

3.4 Checking the data types of the columns for the dataset

- The column range index has 2539 entries (i.e No.of rows = 2539) which starts the number series from 0 to 2538.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2539 entries, 0 to 2538
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Date    2539 non-null   object
 1   Price   2460 non-null   float64
dtypes: float64(1), object(1)
memory usage: 39.8+ KB
```

Table 3 Type of Coulmns for the dataset

3.5 Statistical summary of the dataset

Let's check the statistical summary of the data.

- This table reflects the statistical summary of the data contains various details reflecting as Count, Mean, Std (Standard deviation), Min (Minimum), Max (Maximum), and Percentages (25%, 50%, 75%).

	Price
count	2460.000000
mean	1468.784350
std	282.977384
min	1049.600000
25%	1244.375000
50%	1322.150000
75%	1775.025000
max	2069.400000

Table 4 Statistical summary

3.6 Checking for null values in the data

It contains null values in the dataset as no null values (i.e False).

	0
Date	False
Price	False

dtype: bool

Table 5 Null values in the dataset

3.7 Checking for missing values in the data

The dataset has no missing values.

	0
Date	0
Price	0

dtype: int64

Table 6 Missing values in the data

4. Exploratory Data Analysis

4.1 To find the trend of the data

- It is to create a line plot for 'Price and Date'. We considered Date on x-axis and Price on y-axis.

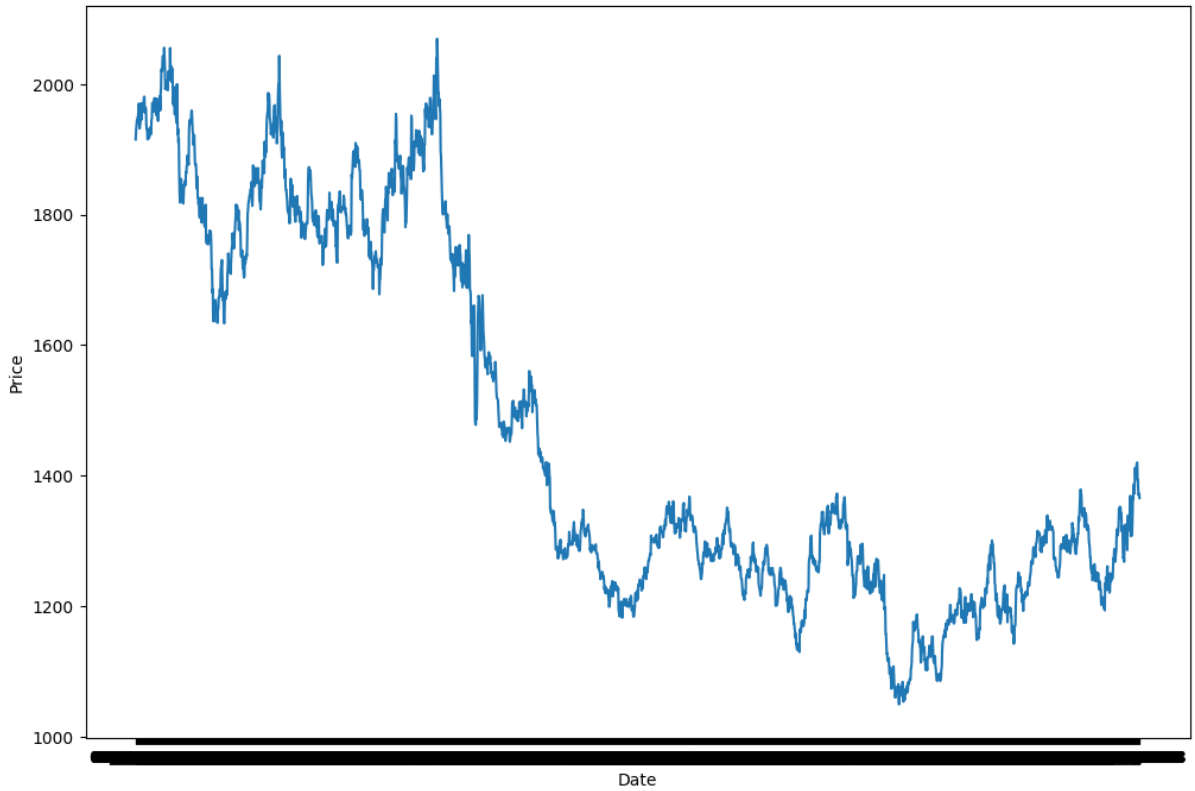

Figure 1 trend of the data

- The line plot showing the gold prices are decreased by increasing the time period.

4.2 Resample the data

- The shape of the data after resampling has rows-121 and columns-2.
  (121, 2)
- We will resample the data to get values at a month-level from the day-level data by taking the average over the months.

Date	index	Price
2013-08-31	2533.5	1391.340000
2013-09-30	2518.0	1348.461905
2013-10-31	2496.0	1316.591304
2013-11-30	2474.0	1273.433333
2013-12-31	2452.5	1223.390909

Table 7 Resample the data

4.3 To find the statistics of the data

- This table reflects the statistical summary of the data after resample contains various details reflecting as Count, Mean, Std (Standard deviation), Min (Minimum), Max (Maximum), and Percentages (25%, 50%, 75%).

	index	Price
count	2539.00000	2460.000000
mean	1269.00000	1468.784350
std	733.09049	282.977384
min	0.00000	1049.600000
25%	634.50000	1244.375000
50%	1269.00000	1322.150000
75%	1903.50000	1775.025000
max	2538.00000	2069.400000

Table 8 Statistical summary of the data

4.4 To find the trend of price at year level

- The trend of price at year level. The price has been reduced from 2013 to 2015 and gradually, increased till 2020. The trend line is continued stable till 2022 then increased later.
- Price of gold is increasing by lagging the time year by year.

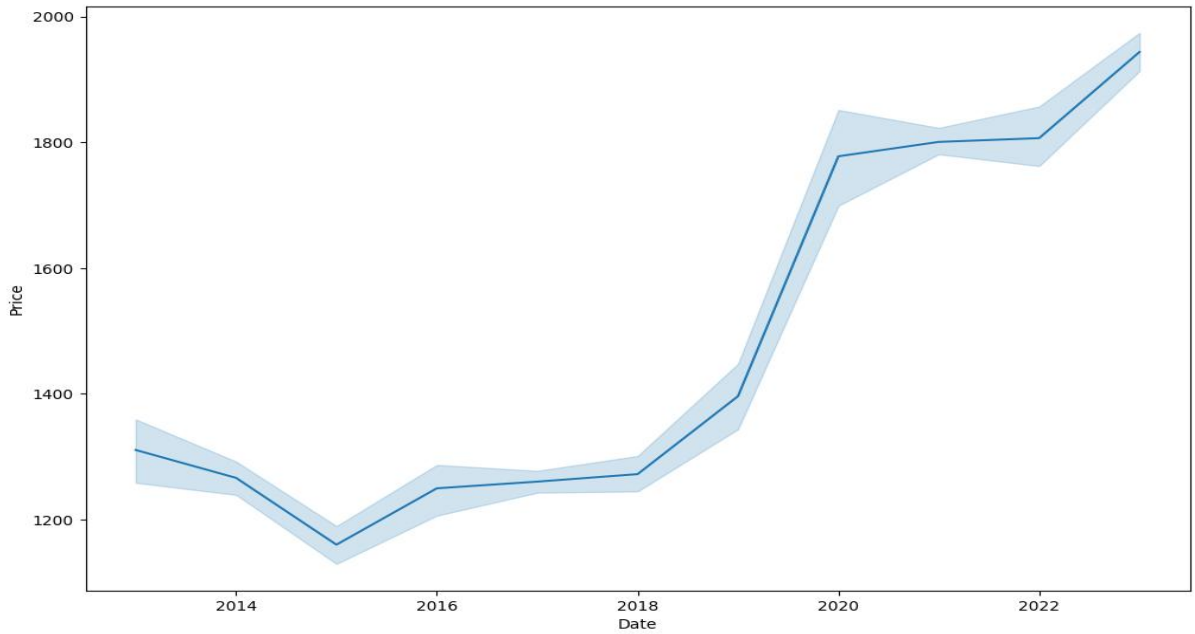

Figure 2 trend of price at year level

4.5 To plot a boxplot at year level

- To plot a boxplot at year level. The mean value of price reduced from 2013 to 2015 then increased continuously till 2020 and the price reached above 1900 in 2023.s

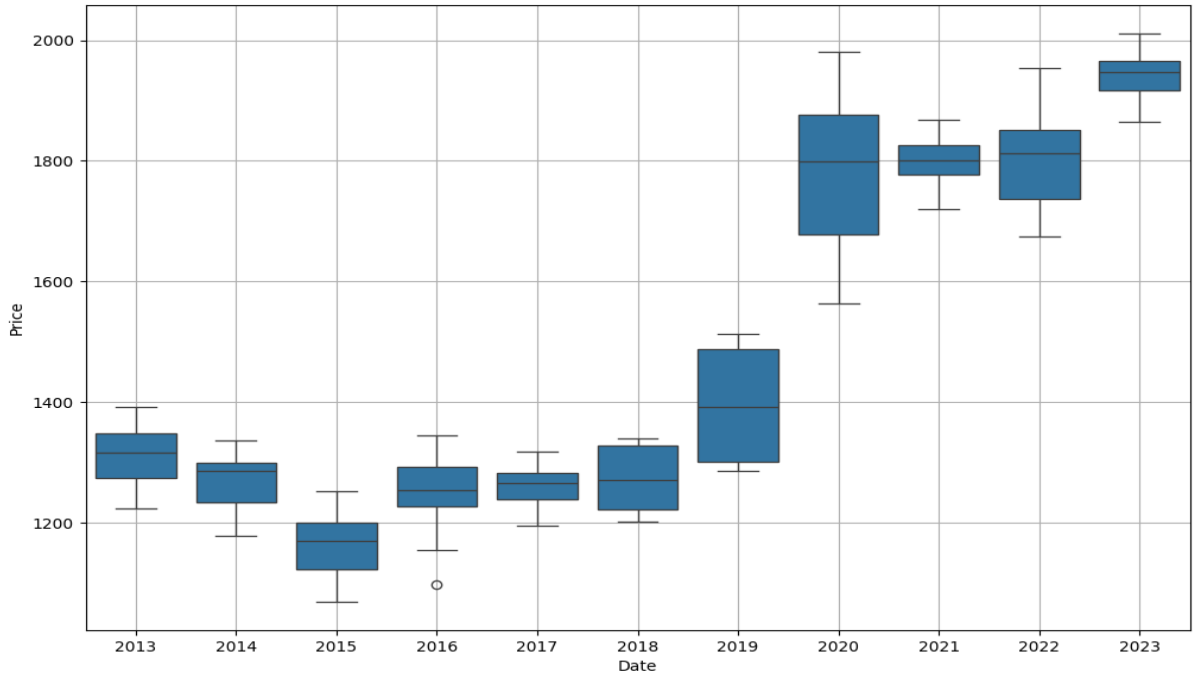

Figure 3 boxplot at year level

4.6 To plot a boxplot at month level

- To plot a boxplot at month level. The boxplot highest price in the month of May and lowest price in October month. January and February months have the price constantly.

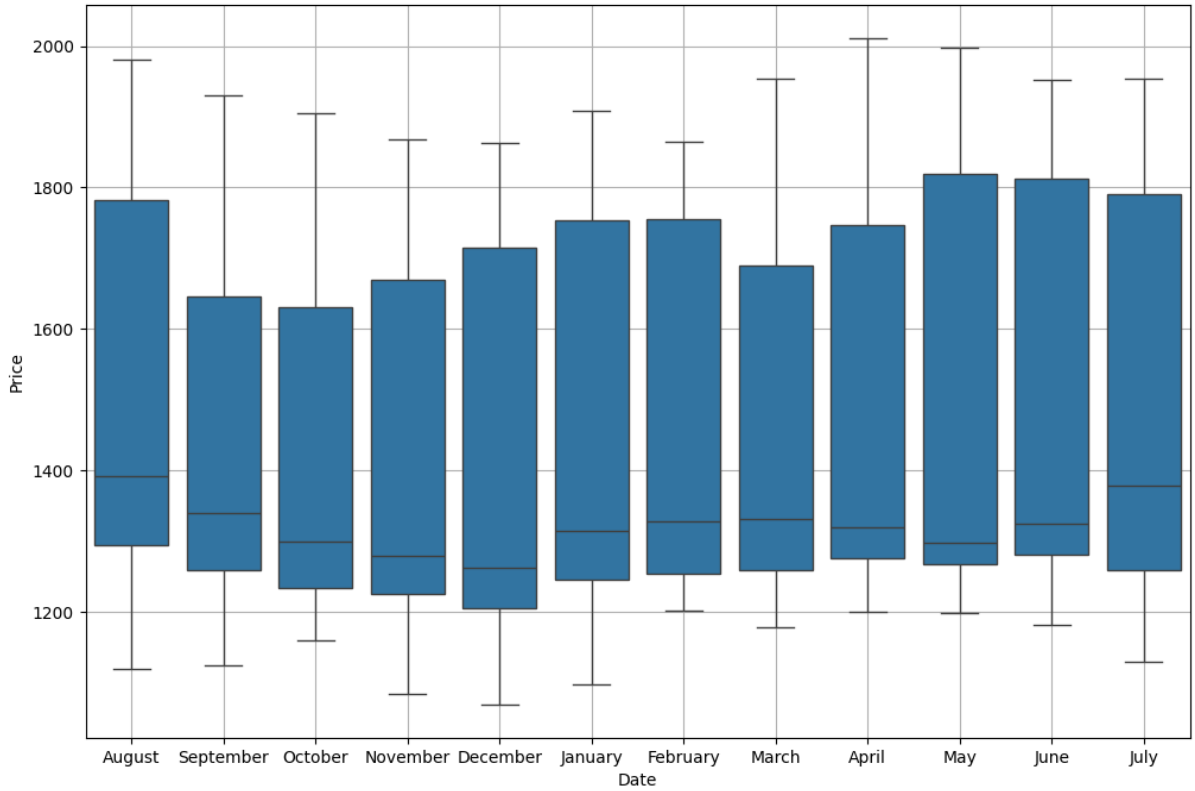

Figure 4 boxplot at month level

4.7 To create a pivot table to display monthly price across years

- The pivot table to display monthly price across years.

Date	April	August	December	February	January	July	June	March	May	November	October	September
Date												
2013	NaN	1391.340000	1223.390909	NaN	NaN	NaN	NaN	NaN	NaN	1273.433333	1316.591304	1348.461905
2014	1298.190909	1296.680952	1199.291304	1302.205000	1242.034783	1311.717391	1282.661905	1335.880952	1287.995455	1177.480000	1223.473913	1237.772727
2015	1200.363636	1118.533333	1068.921739	1225.565000	1252.518182	1129.756522	1181.409091	1178.250000	1198.476190	1083.209524	1159.604545	1124.059091
2016	1243.847619	1344.482609	1153.690476	1201.640000	1097.289474	1337.670000	1279.190909	1245.213636	1260.642857	1235.895238	1265.995238	1330.680952
2017	1271.136842	1290.352174	1268.660000	1236.684211	1194.320000	1238.990000	1262.436364	1231.839130	1245.540909	1283.471429	1281.718182	1318.340000
2018	1338.661905	1205.904348	1256.835000	1332.594737	1334.057143	1239.890476	1283.952381	1326.919048	1305.504545	1222.342857	1218.052174	1201.994737
2019	1288.971429	1511.818182	1484.347619	1322.363158	1294.500000	1418.418182	1363.440000	1301.871429	1286.109091	1470.875000	1497.634783	1513.320000
2020	1703.266667	1980.271429	1862.845455	1599.942105	1562.585714	1855.190909	1743.809091	1596.859091	1724.495000	1867.115000	1905.550000	1929.819048
2021	1760.909524	1786.709091	1792.300000	1807.263158	1867.557895	1807.623810	1835.800000	1719.908696	1851.620000	1822.495238	1777.428571	1776.780952
2022	1937.545000	1778.017391	1807.657143	1859.031579	1817.750000	1738.430000	1838.780952	1953.052174	1849.523810	1734.533333	1673.952381	1690.057143
2023	2011.431579	1941.984615	NaN	1864.415789	1908.955000	1953.925000	1951.438095	1918.621739	1997.940909	NaN	NaN	NaN

Table 9 Pivot table for monthly prices across years

4.8 To plot trend of months across years

- To plot trend of months across years. Overall, the price journey has started at down in 2013 to 2015 then the trend raised till 2020 and it slops upward till 2022. Each month has been represented by different colours.

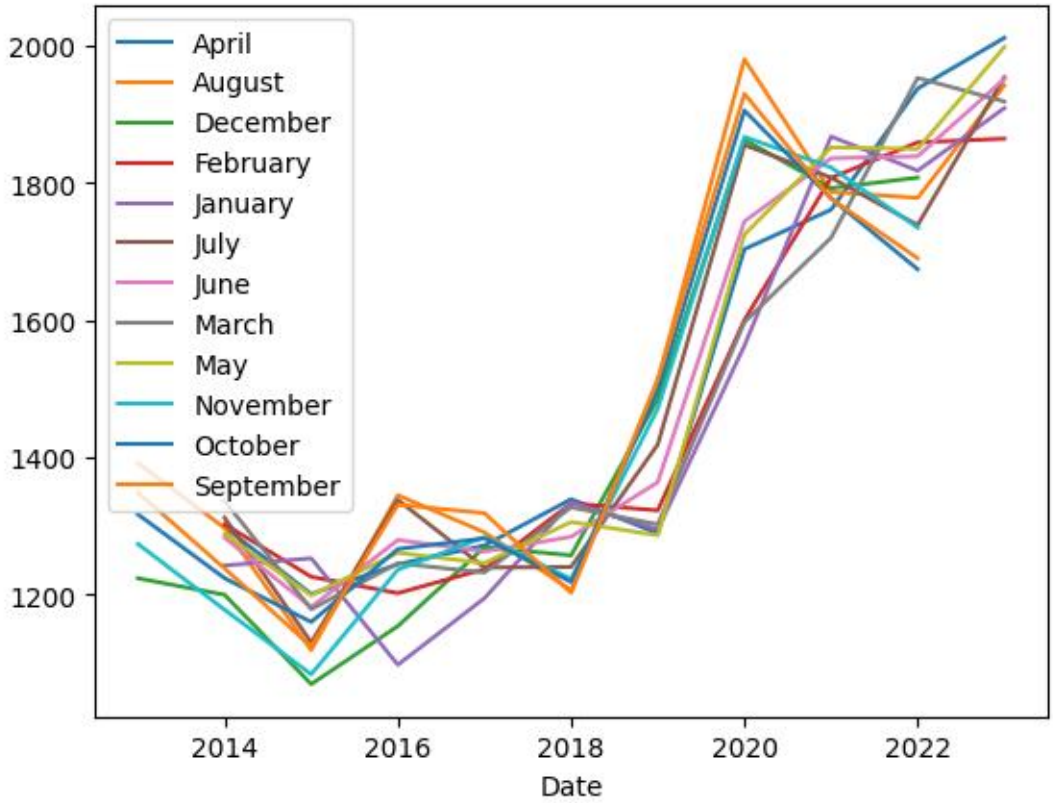

Figure 5 trend of months across years

4.9 Decomposition

Decomposition has included additively and multiplicatively time series.

4.9.1 Decompose the time series additively

Decompose

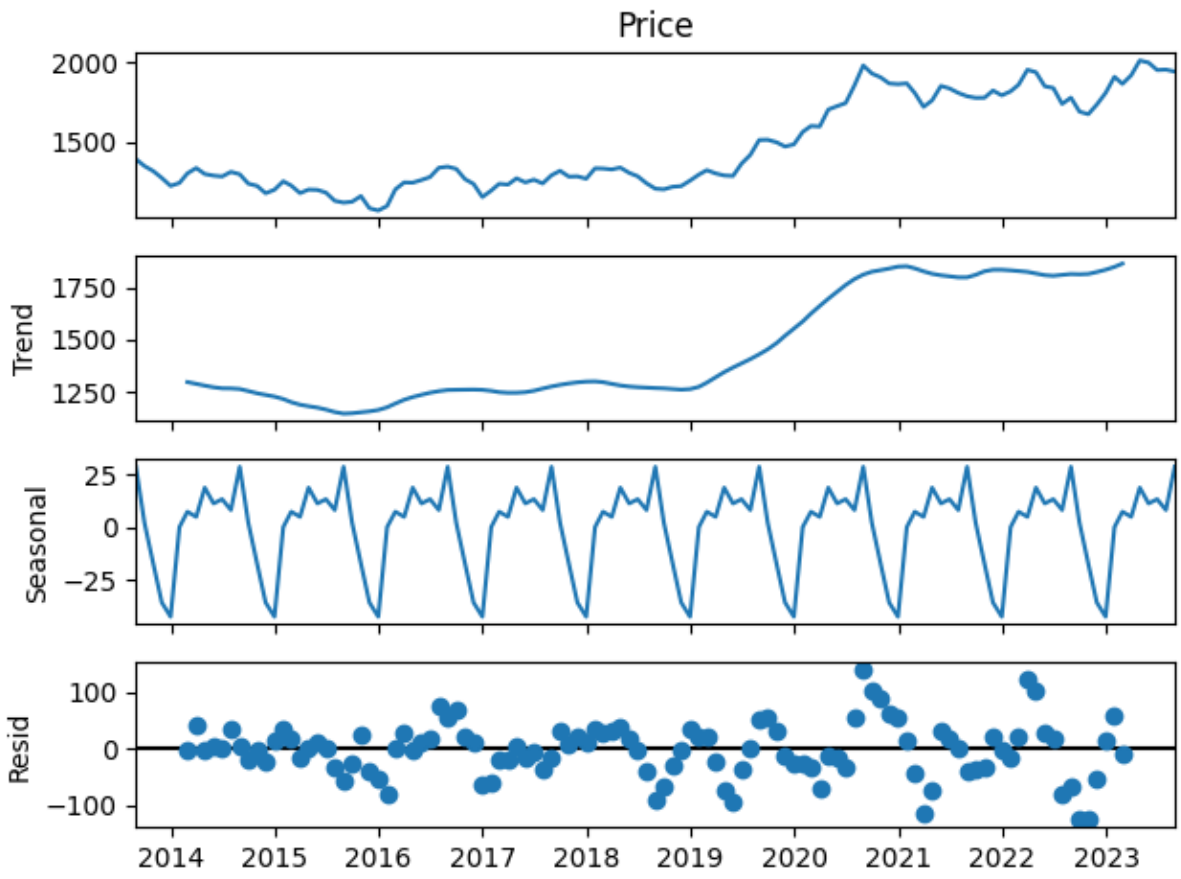

Figure 6 Decompose the time series additively

4.9.2 Decompose the time series multiplicative

To decompose multiplicatively.

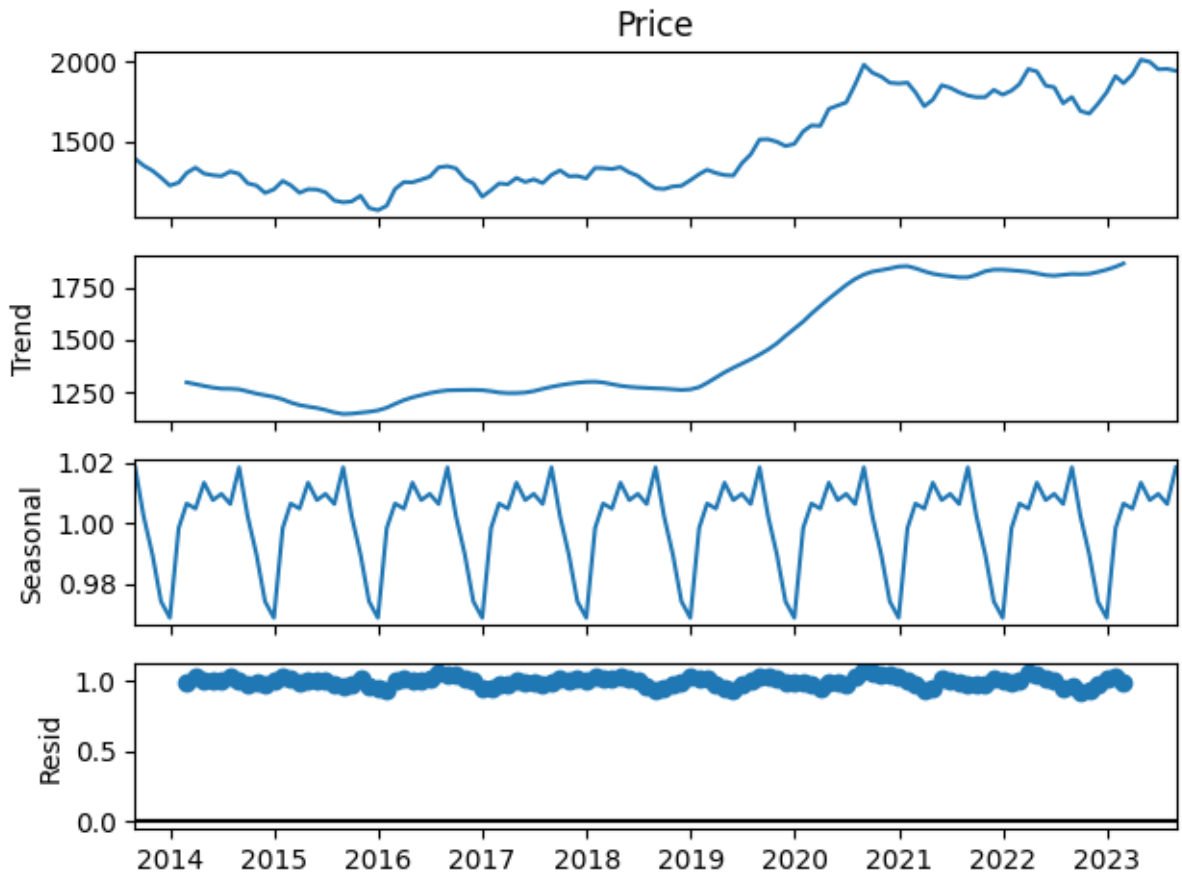

Figure 7 Decompose the time series multiplicative

5. Data Pre-processing

5.1 Split the data into train and test and plot the training and test data

- To split train set consisting data until 2020 and test set from 2021 onwards.
- The below tables contain first 5 rows and last 5 rows of the Training Data.

First few rows of Training Data

Date	index	Price
2013-08-31	2533.5	1391.340000
2013-09-30	2518.0	1348.461905
2013-10-31	2496.0	1316.591304
2013-11-30	2474.0	1273.433333
2013-12-31	2452.5	1223.390909

Last few rows of Training Data

Date	index	Price
2020-08-31	755.0	1980.271429
2020-09-30	734.0	1929.819048
2020-10-31	712.5	1905.550000
2020-11-30	691.5	1867.115000
2020-12-31	670.5	1862.845455

Table 10 First few rows of training data

- To split train set consisting data until 2020 and test set from 2021 onwards.

The below tables contain the first 5 rows and last 5 rows of the Test Data.

```
First few rows of Test Data
```

Date	index	Price
2021-01-31	650.0	1867.557895
2021-02-28	631.0	1807.263158
2021-03-31	610.0	1719.908696
2021-04-30	588.0	1760.909524
2021-05-31	567.5	1851.620000

```
Last few rows of Test Data
```

Date	index	Price
2023-04-30	85.0	2011.431579
2023-05-31	64.5	1997.940909
2023-06-30	43.0	1951.438095
2023-07-31	22.5	1953.925000
2023-08-31	6.0	1941.984615

Table 11 First few rows of test data

5.2 To check the shape of train and test sets

- To check the shape of train and test sets as below.
- Train data has 89 rows and 2 columns.
- Test data – 32 rows and 2 columns.

5.3 To check the shape of train and test sets to find the trend of train data

The trend of train and test data

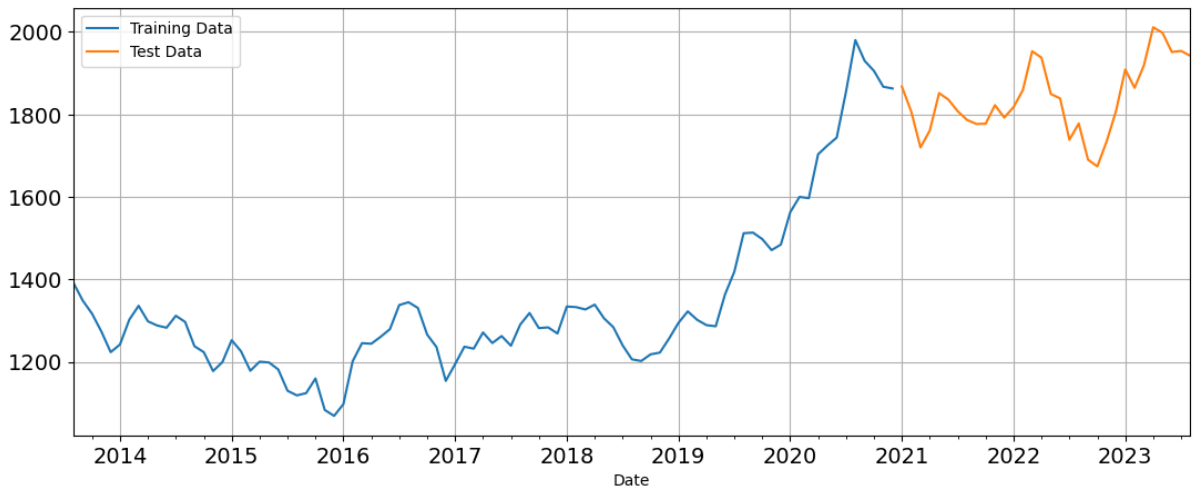

Figure 8 shape/trend of train and test sets

- Training data trend was gone down from 2014 to 2016. Trend raises slightly in 2016 and decreased in 2017. It has been increased gradually from the year 2017 to till 2021.
- Test data trend considered from 2021 to 2023 which is the recent previous years. The trend was increased by passing the years from 2021 (low) to 2023 (high).

6. Model Building- Original Data

The Original data has used to prepare various models like Linear Regression, Moving Average, Simple Exponential Smoothening models, Double Exponential Smoothening models, Triple Exponential Smoothening models.

6.1 Linear Regression

- To generate the numerical time instance order for both the training and test set.
- Training Time instance
- [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89]
- Test Time instance: - It contain the first 5 rows of the data and last 5 rows of the data from both Training and Test data sets.

```
First few rows of Training Data
              index        Price  time
Date
2013-08-31  2533.5  1391.340000     1
2013-09-30  2518.0  1348.461905     2
2013-10-31  2496.0  1316.591304     3
2013-11-30  2474.0  1273.433333     4
2013-12-31  2452.5  1223.390909     5

Last few rows of Training Data
             index        Price  time
Date
2020-08-31  755.0  1980.271429    85
2020-09-30  734.0  1929.819048    86
2020-10-31  712.5  1905.550000    87
2020-11-30  691.5  1867.115000    88
2020-12-31  670.5  1862.845455    89
```

Table 12 First few rows of training data

```
First few rows of Test Data
              index        Price  time
Date
2021-01-31  650.0  1867.557895    89
2021-02-28  631.0  1807.263158    90
2021-03-31  610.0  1719.908696    91
2021-04-30  588.0  1760.909524    92
2021-05-31  567.5  1851.620000    93

Last few rows of Test Data
              index        Price  time
Date
2023-04-30   85.0  2011.431579   116
2023-05-31   64.5  1997.940909   117
2023-06-30   43.0  1951.438095   118
2023-07-31   22.5  1953.925000   119
2023-08-31    6.0  1941.984615   120
```

Table 13 First few rows of test data

6.1.1 Now that our training and test data has been modified, let us go ahead use $LinearRegression$ to build the model on the training data and test the model on the test data.

Build the linear regression model

```
▾ LinearRegression
LinearRegression()
```

Table 14 Linear Regression model

- To make predictions plot the predictions made by the model. The blue line represents the Training data from 2014 to 2021. The orange line shows the Test data and green line indicates the linear regression. The Linear Regression green line has shown the model for actual data.

Figure 9 predictions made by the model

6.1.2 Test Data - RMSE

The RMSE on test data for the RegressionOnTime forecast on the Test Data, RMSE is 207.87.

	Test RMSE
Linear Regression	207.872126

Table 15 LR - Test RMSE

6.2 Moving Average (MA)

For the moving average model, we are going to calculate rolling means (or moving averages) for different intervals. The best interval can be determined by the maximum accuracy (or the minimum error) over here. For Moving Average, we are going to average over the entire data.

	index	Price
Date		
2013-08-31	2533.5	1391.340000
2013-09-30	2518.0	1348.461905
2013-10-31	2496.0	1316.591304
2013-11-30	2474.0	1273.433333
2013-12-31	2452.5	1223.390909

Table 16 Moving Average

6.2.1 Trailing Moving Average

The below table reflecting the four kinds of trailing data for moving average.

	index	Price	Trailing_2	Trailing_4	Trailing_6	Trailing_9
Date						
2013-08-31	2533.5	1391.340000	NaN	NaN	NaN	NaN
2013-09-30	2518.0	1348.461905	1369.900952	NaN	NaN	NaN
2013-10-31	2496.0	1316.591304	1332.526605	NaN	NaN	NaN
2013-11-30	2474.0	1273.433333	1295.012319	1332.456636	NaN	NaN
2013-12-31	2452.5	1223.390909	1248.412121	1290.469363	NaN	NaN

Table 17 Moving Average Trailing

6.2.2 Plotting on the whole data

- All lines in the chart are close to each other. The lines have been plotted from 2014 to 2024 and it is slowly down from 2014 to 2016 then increased slowly and fell down in 2017. These lines are increasing from year 2019 to 2024. It same for all the lines reflecting the chart.

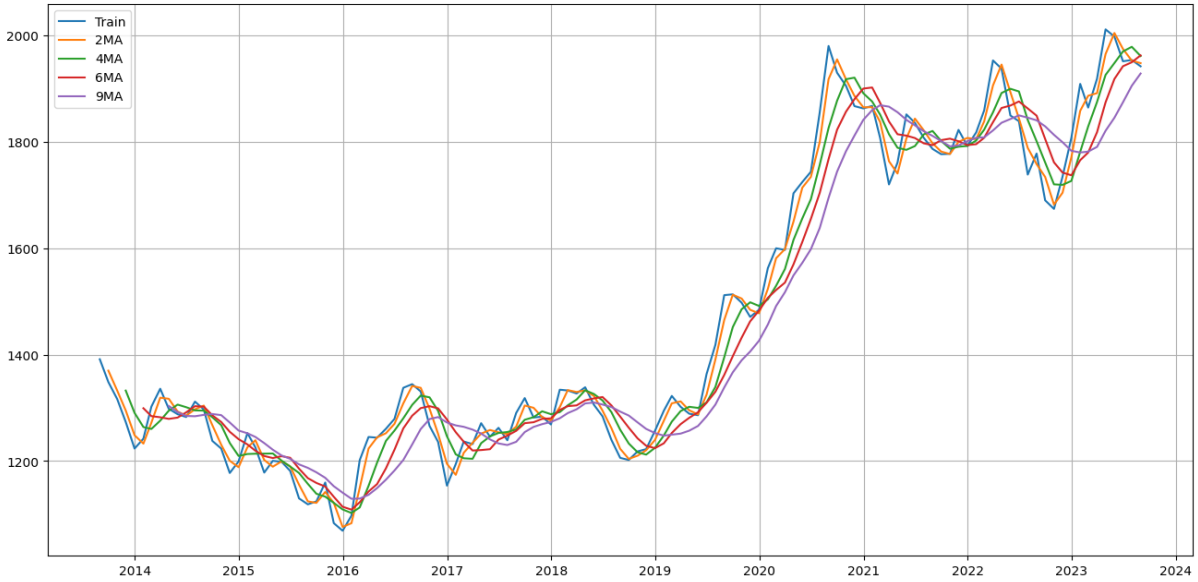

Figure 10 Plotting on the whole data

6.2.3 Let us split the data into train and test and plot this Time Series. The window of the moving average is need to be carefully selected as too big a window will result in not having any test set as the whole series might get averaged over.

- Creating train and test set are in to two categories as below
- Trailing_MovingAverage_train is < 2021 Year and Test_MovingAverage_train is >= 2021 Year.

Test Data – RMSE for different point Trailing MA: -

- For 2 point Moving Average Model forecast on the Testing Data, RMSE is 27.945
- For 4 point Moving Average Model forecast on the Testing Data, RMSE is 54.619
- For 6 point Moving Average Model forecast on the Testing Data, RMSE is 70.894
- For 9 point Moving Average Model forecast on the Testing Data, RMSE is 85.550
- The least Test RMSE – 27.944954 is the model - 2pointTrailingMovingAverage is the best model.

	Test RMSE
Linear Regression	207.872126
2pointTrailingMovingAverage	27.944954
4pointTrailingMovingAverage	54.619271
6pointTrailingMovingAverage	70.894305
9pointTrailingMovingAverage	85.550202

Table 18 LR - Test RMSE

6.2.4 Before we go on to build the various Exponential Smoothing models, let us plot all the best models and compare the Time Series plots.

- Plotting on both Training and Test data for comparison on the chart: -
- to plot the predictions made by the linear regression model and the best moving average model.
- The Training data has represented by blue line, Test line is for orange line, Linear regression line is in green colour and The red colour indicates the 2pointTrailingMovingAverage.

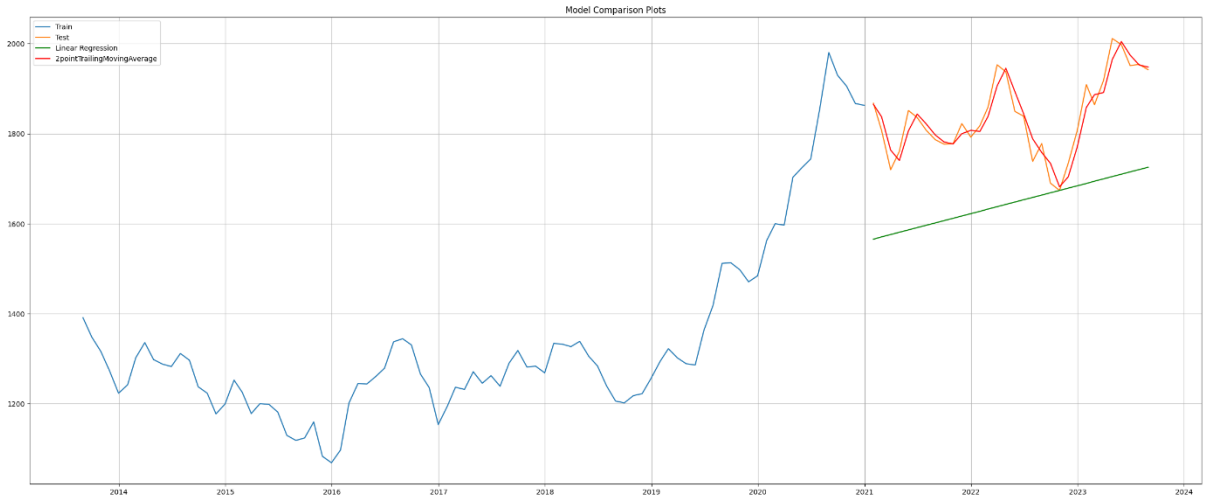

Figure 11 plot all the best models and compare the Time Series plots

6.3 Simple Exponential Smoothening Models

6.3.1 Define the simple exponential smoothening model

Fit the simple exponential smoothening model

```
{'smoothing_level': 0.995,
 'smoothing_trend': nan,
 'smoothing_seasonal': nan,
 'damping_trend': nan,
 'initial_level': 1391.34,
 'initial_trend': nan,
 'initial_seasons': array([], dtype=float64),
 'use_boxcox': False,
 'lamda': None,
 'remove_bias': False}
```

Table 19 Fit the simple exponential smoothening model

6.3.2 Forecast the model

It is forecasting table for simple exponential smoothening model.

	index	Price	predict
Date			
2021-01-31	650.0	1867.557895	1862.867766
2021-02-28	631.0	1807.263158	1862.867766
2021-03-31	610.0	1719.908696	1862.867766
2021-04-30	588.0	1760.909524	1862.867766
2021-05-31	567.5	1851.620000	1862.867766

Table 20 Forecast the model

6.3.3 Plotting on both the Training and Test data

To plot the predictions made by Training data, Test data, the linear regression model, best moving average model and simple exponential smoothening model.

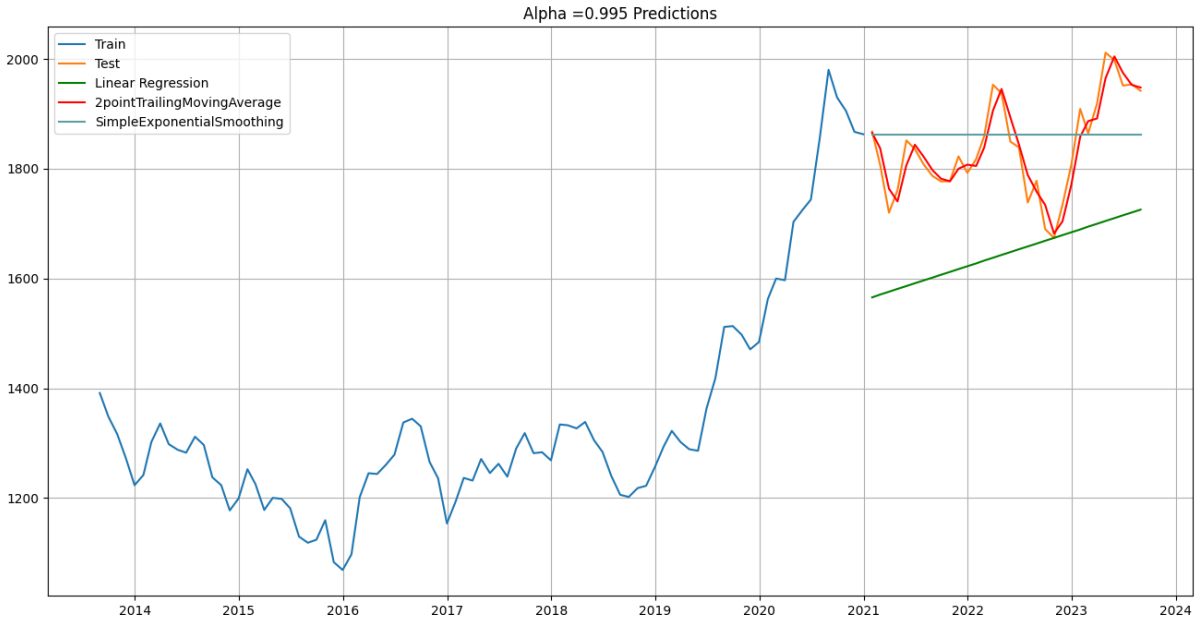

Figure 12 Plotting on both the Training and Test data

6.3.4 Model Evaluation for α = 0.995 : Simple Exponential Smoothing

- Test Data
- to find the RMSE of simple exponential smoothening model
- For Alpha =0.995 Simple Exponential Smoothing Model forecast on the Test Data, RMSE is 89.652
- 2pointTrailingMovingAverage model is the best one which has least Test RMSE.

	Test RMSE
Linear Regression	207.872126
2pointTrailingMovingAverage	27.944954
4pointTrailingMovingAverage	54.619271
6pointTrailingMovingAverage	70.894305
9pointTrailingMovingAverage	85.550202
Alpha=0.995,SimpleExponentialSmoothing	89.651792

Table 21 Simple Exponential Smoothing Model forecast on the Test Data

6.4 Double Exponential Smoothening (Holt's Model)

- Two parameters α and β are estimated in this model. Level and Trend are accounted for in this model.

- Define the double exponential smoothening model. First, we will define an empty dataframe to store our values from the loop.

Alpha Values	Beta Values	Train RMSE	Test RMSE

Table 22 empty dataframe to store our values

6.4.1 fit the model

Forecast the model for RMSE on train data and test data.

	Alpha Values	Beta Values	Train RMSE	Test RMSE
48	0.9	0.3	45.502381	90.076443
56	1.0	0.3	44.429178	90.169207
40	0.8	0.3	47.053203	91.257439
32	0.7	0.3	49.092306	93.551649
24	0.6	0.3	51.726312	113.729176

Table 23 Forecast the model

6.4.2 Plotting on both the Training and Test data

It is to plot the predictions made by the Training data, Test data, linear regression model, best moving average model, simple exponential smoothening model and double exponential smoothening model. The colour wise lines are defined in the legend box reflecting on the below chart.

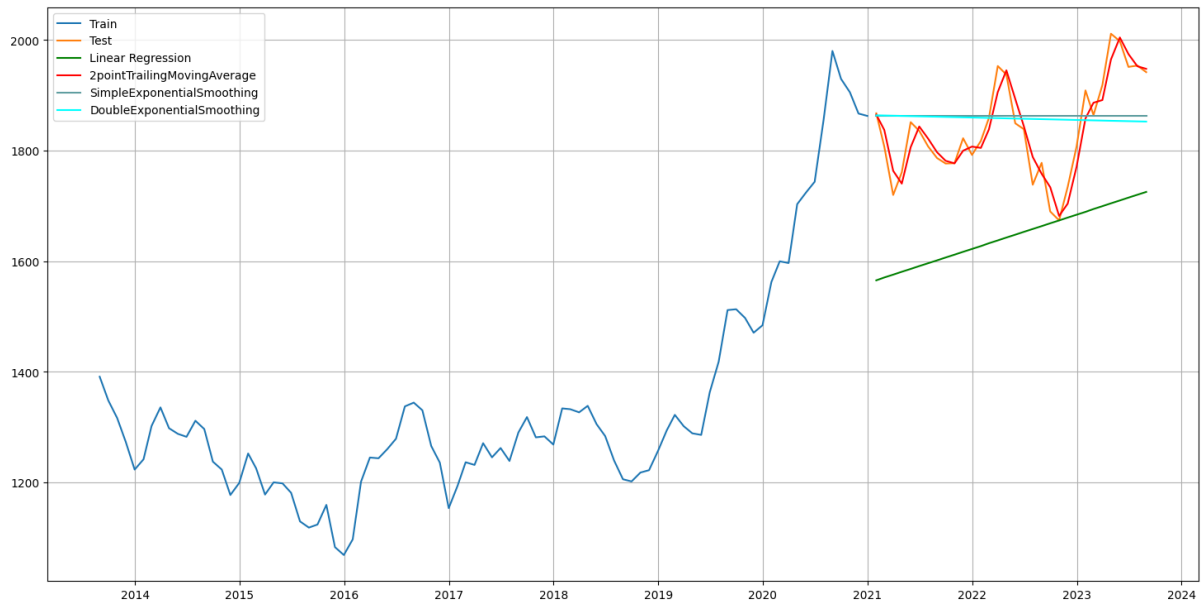

Figure 13 Plotting on both the Training and Test data

The 2pointTrailingMovingAverage model is the best one which has Test RMSE 27.944954.

	Test RMSE
Linear Regression	207.872126
2pointTrailingMovingAverage	27.944954
4pointTrailingMovingAverage	54.619271
6pointTrailingMovingAverage	70.894305
9pointTrailingMovingAverage	85.550202
Alpha=0.995,SimpleExponentialSmoothing	89.651792
Alpha=0.9,Beta=0.3,DoubleExponentialSmoothing	90.076443

Table 24 predictions made by the linear regression model

6.5 Triple Exponential Smoothing (Holt- Winter's Model)

Three parameters α, β and γ are estimated in this model. Level, Trend and Seasonality are accounted for in this model.

6.5.1 Define the triple exponential smoothening model

Fit the triple exponential smoothening model

```
{'smoothing_level': 0.5,
 'smoothing_trend': 0.2,
 'smoothing_seasonal': 0.3,
 'damping_trend': nan,
 'initial_level': 1355.1200447866224,
 'initial_trend': -9.581900788395936,
 'initial_seasons': array([ 43.37398859,  14.48903727,  -1.37672565, -41.39443131,
        -64.52662836, -33.00367542,   3.12618165,  19.02198121,
         24.28527118,  23.64317442,  27.66229978,  38.54266164]),
 'use_boxcox': False,
 'lamda': None,
 'remove_bias': False}
```

Table 25 triple exponential smoothening model

6.5.2 Prediction on the test data

The below is the data to forecast the triple exponential smoothening model

Date	index	Price	auto_predict
2021-01-31	650.0	1867.557895	1949.176367
2021-02-28	631.0	1807.263158	1985.733845
2021-03-31	610.0	1719.908696	1996.440247
2021-04-30	588.0	1760.909524	2056.858415
2021-05-31	567.5	1851.620000	2067.486279

Table 26 Forecast the triple exponential smoothening model

6.5.3 Plotting on both the Training and Test using autofit

- This is to plot the predictions made by the Training, Test data, linear regression model, best moving average model, simple exponential smoothening model, double exponential smoothening model and triple exponential smoothening model.
- The legend box on the chart is describing the colour wise lines respectively. The three exponential smoothing lines are interacting with the actual test data using autofit.

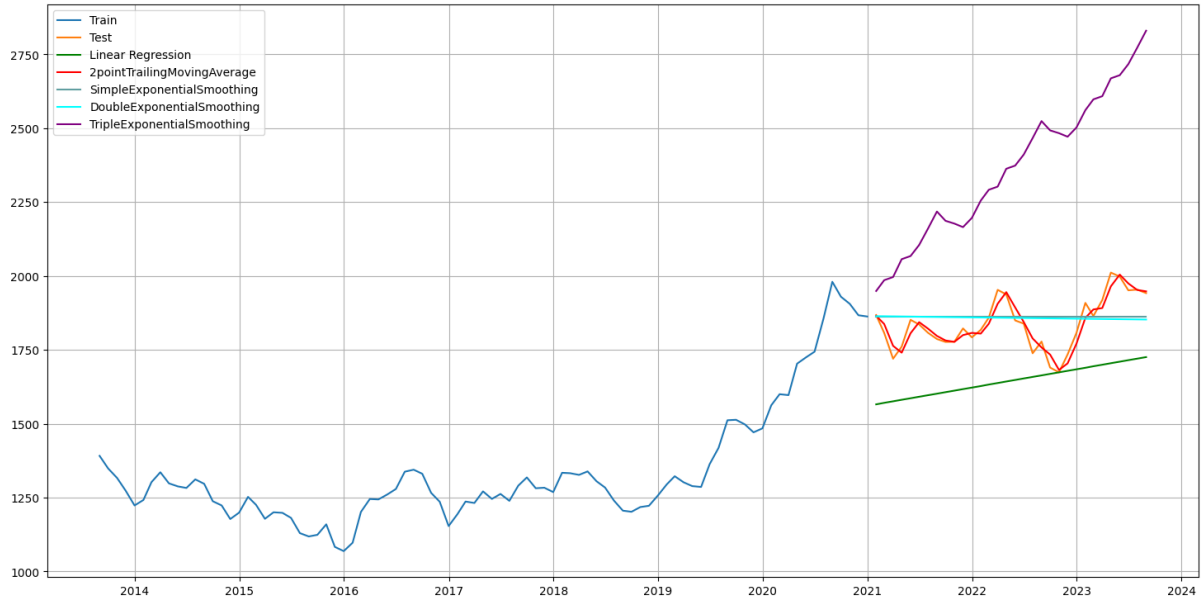

Figure 14 Plotting on both the Training and Test using autofit

6.5.4 Test Data

- The RMSE on test data has calculated for various trailing, Exponential models as reflecting in the table.
- For Alpha=0.676, Beta=0.088, Gamma=0.323, Triple Exponential Smoothing Model forecast on the Test Data, RMSE is 567.793.
- 2pointTrailingMovingAverage is the best model which has least Test RMSE – 27.944954.

	Test RMSE
Linear Regression	207.872126
2pointTrailingMovingAverage	27.944954
4pointTrailingMovingAverage	54.619271
6pointTrailingMovingAverage	70.894305
9pointTrailingMovingAverage	85.550202
Alpha=0.995,SimpleExponentialSmoothing	89.651792
Alpha=0.9,Beta=0.3,DoubleExponentialSmoothing	90.076443
Alpha=0.676,Beta=0.088,Gamma=0.323,TripleExponentialSmoothing	567.793388

Table 27 RMSE on test data

First, the below table define an empty data frame to store our values from the loop.

Alpha Values	Beta Values	Gamma Values	Train RMSE	Test RMSE

Table 28 empty dataframe

6.5.5 Fit the model

The RMSE on train data and test data.

	Alpha Values	Beta Values	Gamma Values	Train RMSE	Test RMSE
346	0.8	0.6	0.5	54.816578	97.653133
338	0.8	0.5	0.5	52.349777	98.271977
297	0.7	0.8	0.4	54.970944	98.645349
396	0.9	0.4	0.7	63.317148	101.824777
288	0.7	0.7	0.3	48.893306	102.127622

Table 29 Fit the model

6.5.6 list (TES_test.columns): -

The below are the list of predictions for different cases of all the values under the columns (index, Price & auto predict).

['index',

'Price',

'auto_predict',

('predict', 0.3, 0.3, 0.3),

('predict', 0.3, 0.3, 0.4),

('predict', 0.3, 0.3, 0.5),

('predict', 0.3, 0.3, 0.6000000000000001),

('predict', 0.3, 0.3, 0.7000000000000002),

('predict', 0.3, 0.3, 0.8000000000000003),

('predict', 0.3, 0.3, 0.9000000000000001),

('predict', 0.3, 0.3, 1.0000000000000002),

('predict', 0.3, 0.4, 0.3),

('predict', 0.3, 0.4, 0.4),

('predict', 0.3, 0.4, 0.5),

('predict', 0.3, 0.4, 0.6000000000000001),

('predict', 0.3, 0.4, 0.7000000000000002),

('predict', 0.3, 0.4, 0.8000000000000003),

('predict', 0.3, 0.4, 0.9000000000000001),

('predict', 0.3, 0.4, 1.0000000000000002),

('predict', 0.3, 0.5, 0.3),

('predict', 0.3, 0.5, 0.4),

('predict', 0.3, 0.5, 0.5),

('predict', 0.3, 0.5, 0.6000000000000001),

('predict', 0.3, 0.5, 0.7000000000000002),

('predict', 0.3, 0.5, 0.8000000000000003),

('predict', 0.3, 0.5, 0.9000000000000001),

('predict', 0.3, 0.5, 1.0000000000000002),

('predict', 0.3, 0.6000000000000001, 0.3),

('predict', 0.3, 0.6000000000000001, 0.4),

('predict', 0.3, 0.6000000000000001, 0.5),

('predict', 0.3, 0.6000000000000001, 0.6000000000000001),

('predict', 0.3, 0.6000000000000001, 0.7000000000000002),

('predict', 0.3, 0.6000000000000001, 0.8000000000000003),

('predict', 0.3, 0.6000000000000001, 0.9000000000000001),

('predict', 0.3, 0.6000000000000001, 1.0000000000000002),

('predict', 0.3, 0.7000000000000002, 0.3),

('predict', 0.3, 0.7000000000000002, 0.4),

('predict', 0.3, 0.7000000000000002, 0.5),

('predict', 0.3, 0.7000000000000002, 0.6000000000000001),

('predict', 0.3, 0.7000000000000002, 0.7000000000000002),

('predict', 0.3, 0.7000000000000002, 0.8000000000000003),

('predict', 0.3, 0.7000000000000002, 0.9000000000000001),

('predict', 0.3, 0.7000000000000002, 1.0000000000000002),

('predict', 0.3, 0.8000000000000003, 0.3),

('predict', 0.3, 0.8000000000000003, 0.4),

('predict', 0.3, 0.8000000000000003, 0.5),

('predict', 0.3, 0.8000000000000003, 0.6000000000000001),

('predict', 0.3, 0.8000000000000003, 0.7000000000000002),

('predict', 0.3, 0.8000000000000003, 0.8000000000000003),

('predict', 0.3, 0.8000000000000003, 0.9000000000000001),

('predict', 0.3, 0.8000000000000003, 1.0000000000000002),

('predict', 0.3, 0.9000000000000001, 0.3),

('predict', 0.3, 0.9000000000000001, 0.4),

('predict', 0.3, 0.9000000000000001, 0.5),

('predict', 0.3, 0.9000000000000001, 0.6000000000000001),

('predict', 0.3, 0.9000000000000001, 0.7000000000000002),

('predict', 0.3, 0.9000000000000001, 0.8000000000000003),

('predict', 0.3, 0.9000000000000001, 0.9000000000000001),

('predict', 0.3, 0.9000000000000001, 1.0000000000000002),

('predict', 0.3, 1.0000000000000002, 0.3),

('predict', 0.3, 1.0000000000000002, 0.4),

('predict', 0.3, 1.0000000000000002, 0.5),

('predict', 0.3, 1.0000000000000002, 0.6000000000000001),

('predict', 0.3, 1.0000000000000002, 0.7000000000000002),

('predict', 0.3, 1.0000000000000002, 0.8000000000000003),

('predict', 0.3, 1.0000000000000002, 0.9000000000000001),

('predict', 0.3, 1.0000000000000002, 1.0000000000000002),

('predict', 0.4, 0.3, 0.3),

('predict', 0.4, 0.3, 0.4),

('predict', 0.4, 0.3, 0.5),

('predict', 0.4, 0.3, 0.6000000000000001),

('predict', 0.4, 0.3, 0.7000000000000002),

('predict', 0.4, 0.3, 0.8000000000000003),

('predict', 0.4, 0.3, 0.9000000000000001),

('predict', 0.4, 0.3, 1.0000000000000002),

('predict', 0.4, 0.4, 0.3),

('predict', 0.4, 0.4, 0.4),

('predict', 0.4, 0.4, 0.5),

('predict', 0.4, 0.4, 0.6000000000000001),

('predict', 0.4, 0.4, 0.7000000000000002),

('predict', 0.4, 0.4, 0.8000000000000003),

('predict', 0.4, 0.4, 0.9000000000000001),

('predict', 0.4, 0.4, 1.0000000000000002),

('predict', 0.4, 0.5, 0.3),

('predict', 0.4, 0.5, 0.4),

('predict', 0.4, 0.5, 0.5),

('predict', 0.4, 0.5, 0.6000000000000001),

('predict', 0.4, 0.5, 0.7000000000000002),

('predict', 0.4, 0.5, 0.8000000000000003),

('predict', 0.4, 0.5, 0.9000000000000001),

('predict', 0.4, 0.5, 1.0000000000000002),

('predict', 0.4, 0.6000000000000001, 0.3),

('predict', 0.4, 0.6000000000000001, 0.4),

('predict', 0.4, 0.6000000000000001, 0.5),

('predict', 0.4, 0.6000000000000001, 0.6000000000000001),

('predict', 0.4, 0.6000000000000001, 0.7000000000000002),

('predict', 0.4, 0.6000000000000001, 0.8000000000000003),

('predict', 0.4, 0.6000000000000001, 0.9000000000000001),

('predict', 0.4, 0.6000000000000001, 1.0000000000000002),

('predict', 0.4, 0.7000000000000002, 0.3),

('predict', 0.4, 0.7000000000000002, 0.4),

('predict', 0.4, 0.7000000000000002, 0.5),

('predict', 0.4, 0.7000000000000002, 0.6000000000000001),

('predict', 0.4, 0.7000000000000002, 0.7000000000000002),

('predict', 0.4, 0.7000000000000002, 0.8000000000000003),

('predict', 0.4, 0.7000000000000002, 0.9000000000000001),

('predict', 0.4, 0.7000000000000002, 1.0000000000000002),

('predict', 0.4, 0.8000000000000003, 0.3),

('predict', 0.4, 0.8000000000000003, 0.4),

('predict', 0.4, 0.8000000000000003, 0.5),

('predict', 0.4, 0.8000000000000003, 0.6000000000000001),

('predict', 0.4, 0.8000000000000003, 0.7000000000000002),

('predict', 0.4, 0.8000000000000003, 0.8000000000000003),

('predict', 0.4, 0.8000000000000003, 0.9000000000000001),

('predict', 0.4, 0.8000000000000003, 1.0000000000000002),

('predict', 0.4, 0.9000000000000001, 0.3),

('predict', 0.4, 0.9000000000000001, 0.4),

('predict', 0.4, 0.9000000000000001, 0.5),

('predict', 0.4, 0.9000000000000001, 0.6000000000000001),

('predict', 0.4, 0.9000000000000001, 0.7000000000000002),

('predict', 0.4, 0.9000000000000001, 0.8000000000000003),

('predict', 0.4, 0.9000000000000001, 0.9000000000000001),

('predict', 0.4, 0.9000000000000001, 1.0000000000000002),

('predict', 0.4, 1.0000000000000002, 0.3),

('predict', 0.4, 1.0000000000000002, 0.4),

('predict', 0.4, 1.0000000000000002, 0.5),

('predict', 0.4, 1.0000000000000002, 0.6000000000000001),

('predict', 0.4, 1.0000000000000002, 0.7000000000000002),

('predict', 0.4, 1.0000000000000002, 0.8000000000000003),

('predict', 0.4, 1.0000000000000002, 0.9000000000000001),

('predict', 0.4, 1.0000000000000002, 1.0000000000000002),

('predict', 0.5, 0.3, 0.3),

('predict', 0.5, 0.3, 0.4),

('predict', 0.5, 0.3, 0.5),

('predict', 0.5, 0.3, 0.6000000000000001),

('predict', 0.5, 0.3, 0.7000000000000002),

('predict', 0.5, 0.3, 0.8000000000000003),

('predict', 0.5, 0.3, 0.9000000000000001),

('predict', 0.5, 0.3, 1.0000000000000002),

('predict', 0.5, 0.4, 0.3),

('predict', 0.5, 0.4, 0.4),

('predict', 0.5, 0.4, 0.5),

('predict', 0.5, 0.4, 0.6000000000000001),

('predict', 0.5, 0.4, 0.7000000000000002),

('predict', 0.5, 0.4, 0.8000000000000003),

('predict', 0.5, 0.4, 0.9000000000000001),

('predict', 0.5, 0.4, 1.0000000000000002),

('predict', 0.5, 0.5, 0.3),

('predict', 0.5, 0.5, 0.4),

('predict', 0.5, 0.5, 0.5),

('predict', 0.5, 0.5, 0.6000000000000001),

('predict', 0.5, 0.5, 0.7000000000000002),

('predict', 0.5, 0.5, 0.8000000000000003),

('predict', 0.5, 0.5, 0.9000000000000001),

('predict', 0.5, 0.5, 1.0000000000000002),

('predict', 0.5, 0.6000000000000001, 0.3),

('predict', 0.5, 0.6000000000000001, 0.4),

('predict', 0.5, 0.6000000000000001, 0.5),

('predict', 0.5, 0.6000000000000001, 0.6000000000000001),

('predict', 0.5, 0.6000000000000001, 0.7000000000000002),

('predict', 0.5, 0.6000000000000001, 0.8000000000000003),

('predict', 0.5, 0.6000000000000001, 0.9000000000000001),

('predict', 0.5, 0.6000000000000001, 1.0000000000000002),

('predict', 0.5, 0.7000000000000002, 0.3),

('predict', 0.5, 0.7000000000000002, 0.4),

('predict', 0.5, 0.7000000000000002, 0.5),

('predict', 0.5, 0.7000000000000002, 0.6000000000000001),

('predict', 0.5, 0.7000000000000002, 0.7000000000000002),

('predict', 0.5, 0.7000000000000002, 0.8000000000000003),

('predict', 0.5, 0.7000000000000002, 0.9000000000000001),

('predict', 0.5, 0.7000000000000002, 1.0000000000000002),

('predict', 0.5, 0.8000000000000003, 0.3),

('predict', 0.5, 0.8000000000000003, 0.4),

('predict', 0.5, 0.8000000000000003, 0.5),

('predict', 0.5, 0.8000000000000003, 0.6000000000000001),

('predict', 0.5, 0.8000000000000003, 0.7000000000000002),

('predict', 0.5, 0.8000000000000003, 0.8000000000000003),

('predict', 0.5, 0.8000000000000003, 0.9000000000000001),

('predict', 0.5, 0.8000000000000003, 1.0000000000000002),

('predict', 0.5, 0.9000000000000001, 0.3),

('predict', 0.5, 0.9000000000000001, 0.4),

('predict', 0.5, 0.9000000000000001, 0.5),

('predict', 0.5, 0.9000000000000001, 0.6000000000000001),

('predict', 0.5, 0.9000000000000001, 0.7000000000000002),

('predict', 0.5, 0.9000000000000001, 0.8000000000000003),

('predict', 0.5, 0.9000000000000001, 0.9000000000000001),

('predict', 0.5, 0.9000000000000001, 1.0000000000000002),

('predict', 0.5, 1.0000000000000002, 0.3),

('predict', 0.5, 1.0000000000000002, 0.4),

('predict', 0.5, 1.0000000000000002, 0.5),

('predict', 0.5, 1.0000000000000002, 0.6000000000000001),

('predict', 0.5, 1.0000000000000002, 0.7000000000000002),

('predict', 0.5, 1.0000000000000002, 0.8000000000000003),

('predict', 0.5, 1.0000000000000002, 0.9000000000000001),

('predict', 0.5, 1.0000000000000002, 1.0000000000000002),

('predict', 0.6000000000000001, 0.3, 0.3),

('predict', 0.6000000000000001, 0.3, 0.4),

('predict', 0.6000000000000001, 0.3, 0.5),

('predict', 0.6000000000000001, 0.3, 0.6000000000000001),

('predict', 0.6000000000000001, 0.3, 0.7000000000000002),

('predict', 0.6000000000000001, 0.3, 0.8000000000000003),

('predict', 0.6000000000000001, 0.3, 0.9000000000000001),

('predict', 0.6000000000000001, 0.3, 1.0000000000000002),

('predict', 0.6000000000000001, 0.4, 0.3),

('predict', 0.6000000000000001, 0.4, 0.4),

('predict', 0.6000000000000001, 0.4, 0.5),

('predict', 0.6000000000000001, 0.4, 0.6000000000000001),

('predict', 0.6000000000000001, 0.4, 0.7000000000000002),

('predict', 0.6000000000000001, 0.4, 0.8000000000000003),

('predict', 0.6000000000000001, 0.4, 0.9000000000000001),

('predict', 0.6000000000000001, 0.4, 1.0000000000000002),

('predict', 0.6000000000000001, 0.5, 0.3),

('predict', 0.6000000000000001, 0.5, 0.4),

('predict', 0.6000000000000001, 0.5, 0.5),

('predict', 0.6000000000000001, 0.5, 0.6000000000000001),

('predict', 0.6000000000000001, 0.5, 0.7000000000000002),

('predict', 0.6000000000000001, 0.5, 0.8000000000000003),

('predict', 0.6000000000000001, 0.5, 0.9000000000000001),

('predict', 0.6000000000000001, 0.5, 1.0000000000000002),

('predict', 0.6000000000000001, 0.6000000000000001, 0.3),

('predict', 0.6000000000000001, 0.6000000000000001, 0.4),

('predict', 0.6000000000000001, 0.6000000000000001, 0.5),

('predict', 0.6000000000000001, 0.6000000000000001, 0.6000000000000001),

('predict', 0.6000000000000001, 0.6000000000000001, 0.7000000000000002),

('predict', 0.6000000000000001, 0.6000000000000001, 0.8000000000000003),

('predict', 0.6000000000000001, 0.6000000000000001, 0.9000000000000001),

('predict', 0.6000000000000001, 0.6000000000000001, 1.0000000000000002),

('predict', 0.6000000000000001, 0.7000000000000002, 0.3),

('predict', 0.6000000000000001, 0.7000000000000002, 0.4),

('predict', 0.6000000000000001, 0.7000000000000002, 0.5),

('predict', 0.6000000000000001, 0.7000000000000002, 0.6000000000000001),

('predict', 0.6000000000000001, 0.7000000000000002, 0.7000000000000002),

('predict', 0.6000000000000001, 0.7000000000000002, 0.8000000000000003),

('predict', 0.6000000000000001, 0.7000000000000002, 0.9000000000000001),

('predict', 0.6000000000000001, 0.7000000000000002, 1.0000000000000002),

('predict', 0.6000000000000001, 0.8000000000000003, 0.3),

('predict', 0.6000000000000001, 0.8000000000000003, 0.4),

('predict', 0.6000000000000001, 0.8000000000000003, 0.5),

('predict', 0.6000000000000001, 0.8000000000000003, 0.6000000000000001),

('predict', 0.6000000000000001, 0.8000000000000003, 0.7000000000000002),

('predict', 0.6000000000000001, 0.8000000000000003, 0.8000000000000003),

('predict', 0.6000000000000001, 0.8000000000000003, 0.9000000000000001),

('predict', 0.6000000000000001, 0.8000000000000003, 1.0000000000000002),

('predict', 0.6000000000000001, 0.9000000000000001, 0.3),

('predict', 0.6000000000000001, 0.9000000000000001, 0.4),

('predict', 0.6000000000000001, 0.9000000000000001, 0.5),

('predict', 0.6000000000000001, 0.9000000000000001, 0.6000000000000001),

('predict', 0.6000000000000001, 0.9000000000000001, 0.7000000000000002),

('predict', 0.6000000000000001, 0.9000000000000001, 0.8000000000000003),

('predict', 0.6000000000000001, 0.9000000000000001, 0.9000000000000001),

('predict', 0.6000000000000001, 0.9000000000000001, 1.0000000000000002),

('predict', 0.6000000000000001, 1.0000000000000002, 0.3),

('predict', 0.6000000000000001, 1.0000000000000002, 0.4),

('predict', 0.6000000000000001, 1.0000000000000002, 0.5),

('predict', 0.6000000000000001, 1.0000000000000002, 0.6000000000000001),

('predict', 0.6000000000000001, 1.0000000000000002, 0.7000000000000002),

('predict', 0.6000000000000001, 1.0000000000000002, 0.8000000000000003),

('predict', 0.6000000000000001, 1.0000000000000002, 0.9000000000000001),

('predict', 0.6000000000000001, 1.0000000000000002, 1.0000000000000002),

('predict', 0.7000000000000002, 0.3, 0.3),

('predict', 0.7000000000000002, 0.3, 0.4),

('predict', 0.7000000000000002, 0.3, 0.5),

('predict', 0.7000000000000002, 0.3, 0.6000000000000001),

('predict', 0.7000000000000002, 0.3, 0.7000000000000002),

('predict', 0.7000000000000002, 0.3, 0.8000000000000003),

('predict', 0.7000000000000002, 0.3, 0.9000000000000001),

('predict', 0.7000000000000002, 0.3, 1.0000000000000002),

('predict', 0.7000000000000002, 0.4, 0.3),

('predict', 0.7000000000000002, 0.4, 0.4),

('predict', 0.7000000000000002, 0.4, 0.5),

('predict', 0.7000000000000002, 0.4, 0.6000000000000001),

('predict', 0.7000000000000002, 0.4, 0.7000000000000002),

('predict', 0.7000000000000002, 0.4, 0.8000000000000003),

('predict', 0.7000000000000002, 0.4, 0.9000000000000001),

('predict', 0.7000000000000002, 0.4, 1.0000000000000002),

('predict', 0.7000000000000002, 0.5, 0.3),

('predict', 0.7000000000000002, 0.5, 0.4),

('predict', 0.7000000000000002, 0.5, 0.5),

('predict', 0.7000000000000002, 0.5, 0.6000000000000001),

('predict', 0.7000000000000002, 0.5, 0.7000000000000002),

('predict', 0.7000000000000002, 0.5, 0.8000000000000003),

('predict', 0.7000000000000002, 0.5, 0.9000000000000001),

('predict', 0.7000000000000002, 0.5, 1.0000000000000002),

('predict', 0.7000000000000002, 0.6000000000000001, 0.3),

('predict', 0.7000000000000002, 0.6000000000000001, 0.4),

('predict', 0.7000000000000002, 0.6000000000000001, 0.5),

('predict', 0.7000000000000002, 0.6000000000000001, 0.6000000000000001),

('predict', 0.7000000000000002, 0.6000000000000001, 0.7000000000000002),

('predict', 0.7000000000000002, 0.6000000000000001, 0.8000000000000003),

('predict', 0.7000000000000002, 0.6000000000000001, 0.9000000000000001),

('predict', 0.7000000000000002, 0.6000000000000001, 1.0000000000000002),

('predict', 0.7000000000000002, 0.7000000000000002, 0.3),

('predict', 0.7000000000000002, 0.7000000000000002, 0.4),

('predict', 0.7000000000000002, 0.7000000000000002, 0.5),

('predict', 0.7000000000000002, 0.7000000000000002, 0.6000000000000001),

('predict', 0.7000000000000002, 0.7000000000000002, 0.7000000000000002),

('predict', 0.7000000000000002, 0.7000000000000002, 0.8000000000000003),

('predict', 0.7000000000000002, 0.7000000000000002, 0.9000000000000001),

('predict', 0.7000000000000002, 0.7000000000000002, 1.0000000000000002),

('predict', 0.7000000000000002, 0.8000000000000003, 0.3),

('predict', 0.7000000000000002, 0.8000000000000003, 0.4),

('predict', 0.7000000000000002, 0.8000000000000003, 0.5),

('predict', 0.7000000000000002, 0.8000000000000003, 0.6000000000000001),

('predict', 0.7000000000000002, 0.8000000000000003, 0.7000000000000002),

('predict', 0.7000000000000002, 0.8000000000000003, 0.8000000000000003),

('predict', 0.7000000000000002, 0.8000000000000003, 0.9000000000000001),

('predict', 0.7000000000000002, 0.8000000000000003, 1.0000000000000002),

('predict', 0.7000000000000002, 0.9000000000000001, 0.3),

('predict', 0.7000000000000002, 0.9000000000000001, 0.4),

('predict', 0.7000000000000002, 0.9000000000000001, 0.5),

('predict', 0.7000000000000002, 0.9000000000000001, 0.6000000000000001),

('predict', 0.7000000000000002, 0.9000000000000001, 0.7000000000000002),

('predict', 0.7000000000000002, 0.9000000000000001, 0.8000000000000003),

('predict', 0.7000000000000002, 0.9000000000000001, 0.9000000000000001),

('predict', 0.7000000000000002, 0.9000000000000001, 1.0000000000000002),

('predict', 0.7000000000000002, 1.0000000000000002, 0.3),

('predict', 0.7000000000000002, 1.0000000000000002, 0.4),

('predict', 0.7000000000000002, 1.0000000000000002, 0.5),

('predict', 0.7000000000000002, 1.0000000000000002, 0.6000000000000001),

('predict', 0.7000000000000002, 1.0000000000000002, 0.7000000000000002),

('predict', 0.7000000000000002, 1.0000000000000002, 0.8000000000000003),

('predict', 0.7000000000000002, 1.0000000000000002, 0.9000000000000001),

('predict', 0.7000000000000002, 1.0000000000000002, 1.0000000000000002),

('predict', 0.8000000000000003, 0.3, 0.3),

('predict', 0.8000000000000003, 0.3, 0.4),

('predict', 0.8000000000000003, 0.3, 0.5),

('predict', 0.8000000000000003, 0.3, 0.6000000000000001),

('predict', 0.8000000000000003, 0.3, 0.7000000000000002),

('predict', 0.8000000000000003, 0.3, 0.8000000000000003),

('predict', 0.8000000000000003, 0.3, 0.9000000000000001),

('predict', 0.8000000000000003, 0.3, 1.0000000000000002),

('predict', 0.8000000000000003, 0.4, 0.3),

('predict', 0.8000000000000003, 0.4, 0.4),

('predict', 0.8000000000000003, 0.4, 0.5),

('predict', 0.8000000000000003, 0.4, 0.6000000000000001),

('predict', 0.8000000000000003, 0.4, 0.7000000000000002),

('predict', 0.8000000000000003, 0.4, 0.8000000000000003),

('predict', 0.8000000000000003, 0.4, 0.9000000000000001),

('predict', 0.8000000000000003, 0.4, 1.0000000000000002),

('predict', 0.8000000000000003, 0.5, 0.3),

('predict', 0.8000000000000003, 0.5, 0.4),

('predict', 0.8000000000000003, 0.5, 0.5),

('predict', 0.8000000000000003, 0.5, 0.6000000000000001),

('predict', 0.8000000000000003, 0.5, 0.7000000000000002),

('predict', 0.8000000000000003, 0.5, 0.8000000000000003),

('predict', 0.8000000000000003, 0.5, 0.9000000000000001),

('predict', 0.8000000000000003, 0.5, 1.0000000000000002),

('predict', 0.8000000000000003, 0.6000000000000001, 0.3),

('predict', 0.8000000000000003, 0.6000000000000001, 0.4),

('predict', 0.8000000000000003, 0.6000000000000001, 0.5),

('predict', 0.8000000000000003, 0.6000000000000001, 0.6000000000000001),

('predict', 0.8000000000000003, 0.6000000000000001, 0.7000000000000002),

('predict', 0.8000000000000003, 0.6000000000000001, 0.8000000000000003),

('predict', 0.8000000000000003, 0.6000000000000001, 0.9000000000000001),

('predict', 0.8000000000000003, 0.6000000000000001, 1.0000000000000002),

('predict', 0.8000000000000003, 0.7000000000000002, 0.3),

('predict', 0.8000000000000003, 0.7000000000000002, 0.4),

('predict', 0.8000000000000003, 0.7000000000000002, 0.5),

('predict', 0.8000000000000003, 0.7000000000000002, 0.6000000000000001),

('predict', 0.8000000000000003, 0.7000000000000002, 0.7000000000000002),

('predict', 0.8000000000000003, 0.7000000000000002, 0.8000000000000003),

('predict', 0.8000000000000003, 0.7000000000000002, 0.9000000000000001),

('predict', 0.8000000000000003, 0.7000000000000002, 1.0000000000000002),

('predict', 0.8000000000000003, 0.8000000000000003, 0.3),

('predict', 0.8000000000000003, 0.8000000000000003, 0.4),

('predict', 0.8000000000000003, 0.8000000000000003, 0.5),

('predict', 0.8000000000000003, 0.8000000000000003, 0.6000000000000001),

('predict', 0.8000000000000003, 0.8000000000000003, 0.7000000000000002),

('predict', 0.8000000000000003, 0.8000000000000003, 0.8000000000000003),

('predict', 0.8000000000000003, 0.8000000000000003, 0.9000000000000001),

('predict', 0.8000000000000003, 0.8000000000000003, 1.0000000000000002),

('predict', 0.8000000000000003, 0.9000000000000001, 0.3),

('predict', 0.8000000000000003, 0.9000000000000001, 0.4),

('predict', 0.8000000000000003, 0.9000000000000001, 0.5),

('predict', 0.8000000000000003, 0.9000000000000001, 0.6000000000000001),

('predict', 0.8000000000000003, 0.9000000000000001, 0.7000000000000002),

('predict', 0.8000000000000003, 0.9000000000000001, 0.8000000000000003),

('predict', 0.8000000000000003, 0.9000000000000001, 0.9000000000000001),

('predict', 0.8000000000000003, 0.9000000000000001, 1.0000000000000002),

('predict', 0.8000000000000003, 1.0000000000000002, 0.3),

('predict', 0.8000000000000003, 1.0000000000000002, 0.4),

('predict', 0.8000000000000003, 1.0000000000000002, 0.5),

('predict', 0.8000000000000003, 1.0000000000000002, 0.6000000000000001),

('predict', 0.8000000000000003, 1.0000000000000002, 0.7000000000000002),

('predict', 0.8000000000000003, 1.0000000000000002, 0.8000000000000003),

('predict', 0.8000000000000003, 1.0000000000000002, 0.9000000000000001),

('predict', 0.8000000000000003, 1.0000000000000002, 1.0000000000000002),

('predict', 0.9000000000000001, 0.3, 0.3),

('predict', 0.9000000000000001, 0.3, 0.4),

('predict', 0.9000000000000001, 0.3, 0.5),

('predict', 0.9000000000000001, 0.3, 0.6000000000000001),

('predict', 0.9000000000000001, 0.3, 0.7000000000000002),

('predict', 0.9000000000000001, 0.3, 0.8000000000000003),

('predict', 0.9000000000000001, 0.3, 0.9000000000000001),

('predict', 0.9000000000000001, 0.3, 1.0000000000000002),

('predict', 0.9000000000000001, 0.4, 0.3),

('predict', 0.9000000000000001, 0.4, 0.4),

('predict', 0.9000000000000001, 0.4, 0.5),

('predict', 0.9000000000000001, 0.4, 0.6000000000000001),

('predict', 0.9000000000000001, 0.4, 0.7000000000000002),

('predict', 0.9000000000000001, 0.4, 0.8000000000000003),

('predict', 0.9000000000000001, 0.4, 0.9000000000000001),

('predict', 0.9000000000000001, 0.4, 1.0000000000000002),

('predict', 0.9000000000000001, 0.5, 0.3),

('predict', 0.9000000000000001, 0.5, 0.4),

('predict', 0.9000000000000001, 0.5, 0.5),

('predict', 0.9000000000000001, 0.5, 0.6000000000000001),

('predict', 0.9000000000000001, 0.5, 0.7000000000000002),

('predict', 0.9000000000000001, 0.5, 0.8000000000000003),

('predict', 0.9000000000000001, 0.5, 0.9000000000000001),

('predict', 0.9000000000000001, 0.5, 1.0000000000000002),

('predict', 0.9000000000000001, 0.6000000000000001, 0.3),

('predict', 0.9000000000000001, 0.6000000000000001, 0.4),

('predict', 0.9000000000000001, 0.6000000000000001, 0.5),

('predict', 0.9000000000000001, 0.6000000000000001, 0.6000000000000001),

('predict', 0.9000000000000001, 0.6000000000000001, 0.7000000000000002),

('predict', 0.9000000000000001, 0.6000000000000001, 0.8000000000000003),

('predict', 0.9000000000000001, 0.6000000000000001, 0.9000000000000001),

('predict', 0.9000000000000001, 0.6000000000000001, 1.0000000000000002),

('predict', 0.9000000000000001, 0.7000000000000002, 0.3),

('predict', 0.9000000000000001, 0.7000000000000002, 0.4),

('predict', 0.9000000000000001, 0.7000000000000002, 0.5),

('predict', 0.9000000000000001, 0.7000000000000002, 0.6000000000000001),

('predict', 0.9000000000000001, 0.7000000000000002, 0.7000000000000002),

('predict', 0.9000000000000001, 0.7000000000000002, 0.8000000000000003),

('predict', 0.9000000000000001, 0.7000000000000002, 0.9000000000000001),

('predict', 0.9000000000000001, 0.7000000000000002, 1.0000000000000002),

('predict', 0.9000000000000001, 0.8000000000000003, 0.3),

('predict', 0.9000000000000001, 0.8000000000000003, 0.4),

('predict', 0.9000000000000001, 0.8000000000000003, 0.5),

('predict', 0.9000000000000001, 0.8000000000000003, 0.6000000000000001),

('predict', 0.9000000000000001, 0.8000000000000003, 0.7000000000000002),

('predict', 0.9000000000000001, 0.8000000000000003, 0.8000000000000003),

('predict', 0.9000000000000001, 0.8000000000000003, 0.9000000000000001),

('predict', 0.9000000000000001, 0.8000000000000003, 1.0000000000000002),

('predict', 0.9000000000000001, 0.9000000000000001, 0.3),

('predict', 0.9000000000000001, 0.9000000000000001, 0.4),

('predict', 0.9000000000000001, 0.9000000000000001, 0.5),

('predict', 0.9000000000000001, 0.9000000000000001, 0.6000000000000001),

('predict', 0.9000000000000001, 0.9000000000000001, 0.7000000000000002),

('predict', 0.9000000000000001, 0.9000000000000001, 0.8000000000000003),

('predict', 0.9000000000000001, 0.9000000000000001, 0.9000000000000001),

('predict', 0.9000000000000001, 0.9000000000000001, 1.0000000000000002),

('predict', 0.9000000000000001, 1.0000000000000002, 0.3),

('predict', 0.9000000000000001, 1.0000000000000002, 0.4),

('predict', 0.9000000000000001, 1.0000000000000002, 0.5),

('predict', 0.9000000000000001, 1.0000000000000002, 0.6000000000000001),

('predict', 0.9000000000000001, 1.0000000000000002, 0.7000000000000002),

('predict', 0.9000000000000001, 1.0000000000000002, 0.8000000000000003),

('predict', 0.9000000000000001, 1.0000000000000002, 0.9000000000000001),

('predict', 0.9000000000000001, 1.0000000000000002, 1.0000000000000002),

('predict', 1.0000000000000002, 0.3, 0.3),

('predict', 1.0000000000000002, 0.3, 0.4),

('predict', 1.0000000000000002, 0.3, 0.5),

('predict', 1.0000000000000002, 0.3, 0.6000000000000001),

('predict', 1.0000000000000002, 0.3, 0.7000000000000002),

('predict', 1.0000000000000002, 0.3, 0.8000000000000003),

('predict', 1.0000000000000002, 0.3, 0.9000000000000001),

('predict', 1.0000000000000002, 0.3, 1.0000000000000002),

('predict', 1.0000000000000002, 0.4, 0.3),

('predict', 1.0000000000000002, 0.4, 0.4),

('predict', 1.0000000000000002, 0.4, 0.5),

('predict', 1.0000000000000002, 0.4, 0.6000000000000001),

('predict', 1.0000000000000002, 0.4, 0.7000000000000002),

('predict', 1.0000000000000002, 0.4, 0.8000000000000003),

('predict', 1.0000000000000002, 0.4, 0.9000000000000001),

('predict', 1.0000000000000002, 0.4, 1.0000000000000002),

('predict', 1.0000000000000002, 0.5, 0.3),

('predict', 1.0000000000000002, 0.5, 0.4),

('predict', 1.0000000000000002, 0.5, 0.5),

('predict', 1.0000000000000002, 0.5, 0.6000000000000001),

('predict', 1.0000000000000002, 0.5, 0.7000000000000002),

('predict', 1.0000000000000002, 0.5, 0.8000000000000003),

('predict', 1.0000000000000002, 0.5, 0.9000000000000001),

('predict', 1.0000000000000002, 0.5, 1.0000000000000002),

('predict', 1.0000000000000002, 0.6000000000000001, 0.3),

('predict', 1.0000000000000002, 0.6000000000000001, 0.4),

('predict', 1.0000000000000002, 0.6000000000000001, 0.5),

('predict', 1.0000000000000002, 0.6000000000000001, 0.6000000000000001),

('predict', 1.0000000000000002, 0.6000000000000001, 0.7000000000000002),

('predict', 1.0000000000000002, 0.6000000000000001, 0.8000000000000003),

('predict', 1.0000000000000002, 0.6000000000000001, 0.9000000000000001),

('predict', 1.0000000000000002, 0.6000000000000001, 1.0000000000000002),

('predict', 1.0000000000000002, 0.7000000000000002, 0.3),

('predict', 1.0000000000000002, 0.7000000000000002, 0.4),

('predict', 1.0000000000000002, 0.7000000000000002, 0.5),

('predict', 1.0000000000000002, 0.7000000000000002, 0.6000000000000001),

('predict', 1.0000000000000002, 0.7000000000000002, 0.7000000000000002),

('predict', 1.0000000000000002, 0.7000000000000002, 0.8000000000000003),

('predict', 1.0000000000000002, 0.7000000000000002, 0.9000000000000001),

('predict', 1.0000000000000002, 0.7000000000000002, 1.0000000000000002),

('predict', 1.0000000000000002, 0.8000000000000003, 0.3),

('predict', 1.0000000000000002, 0.8000000000000003, 0.4),

('predict', 1.0000000000000002, 0.8000000000000003, 0.5),

('predict', 1.0000000000000002, 0.8000000000000003, 0.6000000000000001),

('predict', 1.0000000000000002, 0.8000000000000003, 0.7000000000000002),

('predict', 1.0000000000000002, 0.8000000000000003, 0.8000000000000003),

('predict', 1.0000000000000002, 0.8000000000000003, 0.9000000000000001),

('predict', 1.0000000000000002, 0.8000000000000003, 1.0000000000000002),

('predict', 1.0000000000000002, 0.9000000000000001, 0.3),

('predict', 1.0000000000000002, 0.9000000000000001, 0.4),

('predict', 1.0000000000000002, 0.9000000000000001, 0.5),

('predict', 1.0000000000000002, 0.9000000000000001, 0.6000000000000001),

('predict', 1.0000000000000002, 0.9000000000000001, 0.7000000000000002),

('predict', 1.0000000000000002, 0.9000000000000001, 0.8000000000000003),

('predict', 1.0000000000000002, 0.9000000000000001, 0.9000000000000001),

('predict', 1.0000000000000002, 0.9000000000000001, 1.0000000000000002),

('predict', 1.0000000000000002, 1.0000000000000002, 0.3),

('predict', 1.0000000000000002, 1.0000000000000002, 0.4),

('predict', 1.0000000000000002, 1.0000000000000002, 0.5),

('predict', 1.0000000000000002, 1.0000000000000002, 0.6000000000000001),

('predict', 1.0000000000000002, 1.0000000000000002, 0.7000000000000002),

('predict', 1.0000000000000002, 1.0000000000000002, 0.8000000000000003),

('predict', 1.0000000000000002, 1.0000000000000002, 0.9000000000000001),

('predict', 1.0000000000000002, 1.0000000000000002, 1.0000000000000002)]

6.5.7 Plotting on both the Training and Test data using brute force alpha, beta and gamma determination

This is to plot the predictions made by the linear regression model, best moving average model, simple exponential smoothening model, double exponential smoothening model and triple exponential smoothening model.

Figure 15 Plotting on both the Training and Test data using brute force alpha, beta and gamma determination

Comparison of Linear Regression, Different pointTrailing and ExponentialSmoothing models.

	Test RMSE
Linear Regression	207.872126
2pointTrailingMovingAverage	27.944954
4pointTrailingMovingAverage	54.619271
6pointTrailingMovingAverage	70.894305
9pointTrailingMovingAverage	85.550202
Alpha=0.995,SimpleExponentialSmoothing	89.651792
Alpha=0.9,Beta=0.3,DoubleExponentialSmoothing	90.076443
Alpha=0.676,Beta=0.088,Gamma=0.323,TripleExponentialSmoothing	567.793388
Alpha=0.8,Beta=0.5,Gamma=0.5,TripleExponentialSmoothing	97.653133

Table 30 Training and Test data using brute force alpha, beta and gamma determination

6.5.7 Sorted by RMSE values on the Test Data:

The below models are sorted by Test RMSE in descending order.

	Test RMSE
2pointTrailingMovingAverage	27.944954
4pointTrailingMovingAverage	54.619271
6pointTrailingMovingAverage	70.894305
9pointTrailingMovingAverage	85.550202
Alpha=0.995,SimpleExponentialSmoothing	89.651792
Alpha=0.9,Beta=0.3,DoubleExponentialSmoothing	90.076443
Alpha=0.8,Beta=0.5,Gamma=0.5,TripleExponentialSmoothing	97.653133
Linear Regression	207.872126
Alpha=0.676,Beta=0.088,Gamma=0.323,TripleExponentialSmoothing	567.793388

Table 31 RMSE values on the Test Data

6.5.8 Test for stationarity of the series - Dicky Fuller test

- Determining rolling statistics, determining the rolling mean and determining the rolling standard deviation.
- Perform Dickey-Fuller test to test stationarity on the data.
- The blue line defined the original data, red line is for Rolling Mean and Black line is for Rolling Std (Standard Deviation).

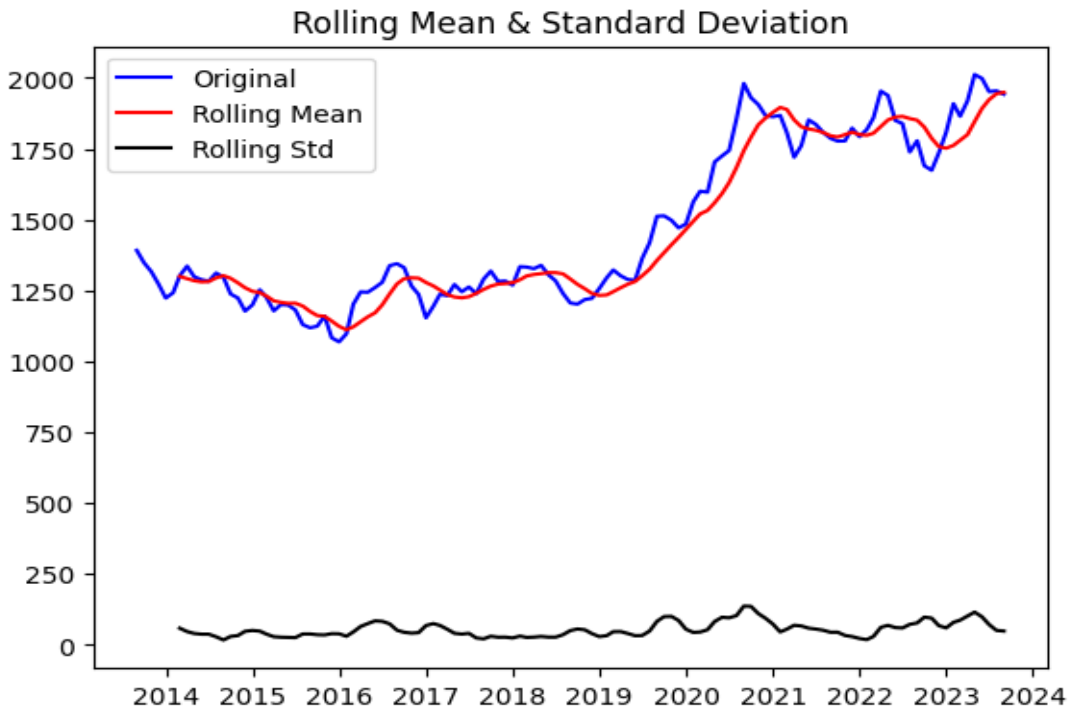

Figure 16 stationarity of the series - Dicky Fuller test

Results of Dickey-Fuller Test: -

P-value (0.877705) is greater than 0.05 in this Results of Dickey – Fuller Test so the time series is non stationary. So, it needs to perform the test again.

The Test statistics (-0.569598) greater than critical values so the null hypothesis is rejected and the time series is non stationary.

```
Results of Dickey-Fuller Test:
Test Statistic                  -0.569598
p-value                          0.877705
#Lags Used                       1.000000
Number of Observations Used    119.000000
Critical Value (1%)             -3.486535
Critical Value (5%)             -2.886151
Critical Value (10%)            -2.579896
dtype: float64
```

Table 32 stationarity of the series - Dicky Fuller test

To perform differencing on train data

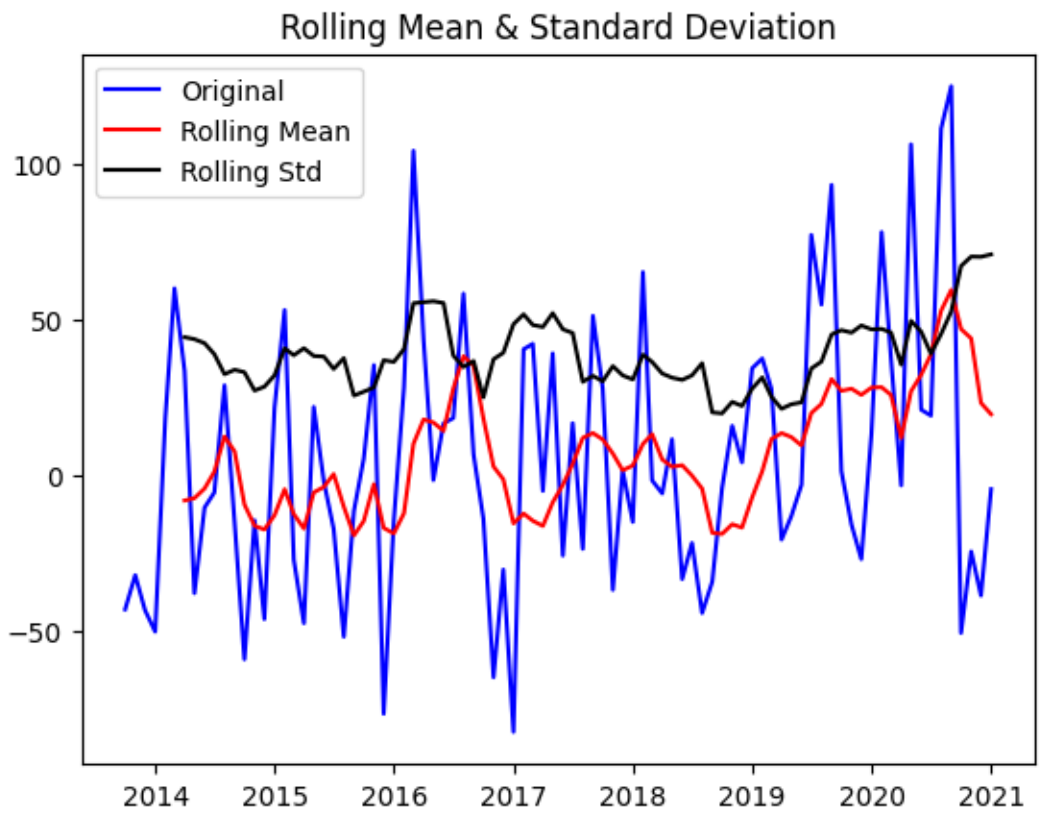

Figure 17 differencing on train data

Results of Dickey-Fuller Test: -

P-value (8.872747e-10) is less than 0.05 in the Results of Dickey – Fuller Test so the time series is stationary.

The Test statistics has more negative (-6.966787e+00) than critical values so the time series is stationary.

```
Test Statistic                  -6.966787e+00
p-value                          8.872747e-10
#Lags Used                       0.000000e+00
Number of Observations Used      8.700000e+01
Critical Value (1%)             -3.507853e+00
Critical Value (5%)             -2.895382e+00
Critical Value (10%)            -2.584824e+00
dtype: float64
```

Table 33 perform differencing on train data

7. Model Building- Stationary Data

7.1 Plot the Autocorrelation and the Partial Autocorrelation function plots on the whole data.

It is to plot ACF (Autocorrelation) plot and PACF (Partial Autocorrelation) plot.

7.2 Auto ARIMA Model

- The following loop helps us in getting a combination of different parameters of p and q in the range of 0 and 2
- We have kept the value of d as 1 as we need to take a difference of the series to make it stationary.
- to ensure stationarity
- Some parameter combinations for the Model...

```
Some parameter combinations for the Model...
Model: (0, 1, 1)
Model: (0, 1, 2)
Model: (1, 1, 0)
Model: (1, 1, 1)
Model: (1, 1, 2)
Model: (2, 1, 0)
Model: (2, 1, 1)
Model: (2, 1, 2)
```

Table 34 Auto ARIMA Model

7.2.1 Creating an empty Dataframe with column names only

param	AIC

Table 35 empty Dataframe

7.2.2 Fit the ARIMA model

It is to fit the ARIMA Model to find the AIC values

```
ARIMA(0, 1, 0) - AIC:914.2597353789228
ARIMA(0, 1, 1) - AIC:908.1514792211835
ARIMA(0, 1, 2) - AIC:909.7965186418962
ARIMA(1, 1, 0) - AIC:908.4009658697732
ARIMA(1, 1, 1) - AIC:909.950670899301
ARIMA(1, 1, 2) - AIC:911.3728643896695
ARIMA(2, 1, 0) - AIC:909.6915224955326
ARIMA(2, 1, 1) - AIC:911.5729813396761
ARIMA(2, 1, 2) - AIC:912.9244622316618
```

Table 36 ARIMA model AIC Values

7.2.3 Sort the above AIC values in the ascending order to get the parameters for the minimum AIC value

The minimum Param (0, 1, 1) is best param which has lowest AIC value (908.151479).

	param	AIC
1	(0, 1, 1)	908.151479
3	(1, 1, 0)	908.400966
6	(2, 1, 0)	909.691522
2	(0, 1, 2)	909.796519
4	(1, 1, 1)	909.950671
5	(1, 1, 2)	911.372864
7	(2, 1, 1)	911.572981
8	(2, 1, 2)	912.924462
0	(0, 1, 0)	914.259735

Table 37 parameters for the minimum AIC value

7.2.4 Sort the above AIC values in the ascending order to get the parameters for the minimum AIC value

- Assign the best parameter to the variable best_param
- Build the ARIMA model based on the best AIC values
- Fit the auto ARIMA model and the summary result
  Best param: (0, 1, 1)

```
Best param: (0, 1, 1)
                               SARIMAX Results
==============================================================================
Dep. Variable:                  Price   No. Observations:                   89
Model:                 ARIMA(0, 1, 1)   Log Likelihood                -452.076
Date:                Thu, 13 Feb 2025   AIC                            908.151
Time:                        16:25:23   BIC                            913.106
Sample:                    08-31-2013   HQIC                           910.148
                         - 12-31-2020
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ma.L1          0.2937      0.111      2.637      0.008       0.075       0.512
sigma2      1694.5828    243.026      6.973      0.000    1218.260    2170.905
===================================================================================
Ljung-Box (L1) (Q):                   0.01   Jarque-Bera (JB):                 2.45
Prob(Q):                              0.94   Prob(JB):                         0.29
Heteroskedasticity (H):               1.79   Skew:                             0.41
Prob(H) (two-sided):                  0.12   Kurtosis:                         3.09
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

Table 38 parameters for the minimum AIC value

7.2.5 Predict on the Test Set using this model and evaluate the model.

Forecast using the auto ARIMA and below values are the predicted mean values.

	predicted_mean
2021-01-31	1864.874169
2021-02-28	1864.874169
2021-03-31	1864.874169
2021-04-30	1864.874169
2021-05-31	1864.874169
2021-06-30	1864.874169
2021-07-31	1864.874169
2021-08-31	1864.874169
2021-09-30	1864.874169
2021-10-31	1864.874169
2021-11-30	1864.874169
2021-12-31	1864.874169
2022-01-31	1864.874169

2022-02-28	1864.874169
2022-03-31	1864.874169
2022-04-30	1864.874169
2022-05-31	1864.874169
2022-06-30	1864.874169
2022-07-31	1864.874169
2022-08-31	1864.874169
2022-09-30	1864.874169
2022-10-31	1864.874169
2022-11-30	1864.874169
2022-12-31	1864.874169
2023-01-31	1864.874169
2023-02-28	1864.874169
2023-03-31	1864.874169
2023-04-30	1864.874169
2023-05-31	1864.874169
2023-06-30	1864.874169
2023-07-31	1864.874169
2023-08-31	1864.874169

dtype: float64

7.2.6 Plotting on both the Training and Test using autofit

The Training and Test using autofit to plot the predictions made by the linear regression model, best moving average model, simple exponential smoothening model, double exponential smoothening model, triple exponential smoothening model and Auto ARIMA model.

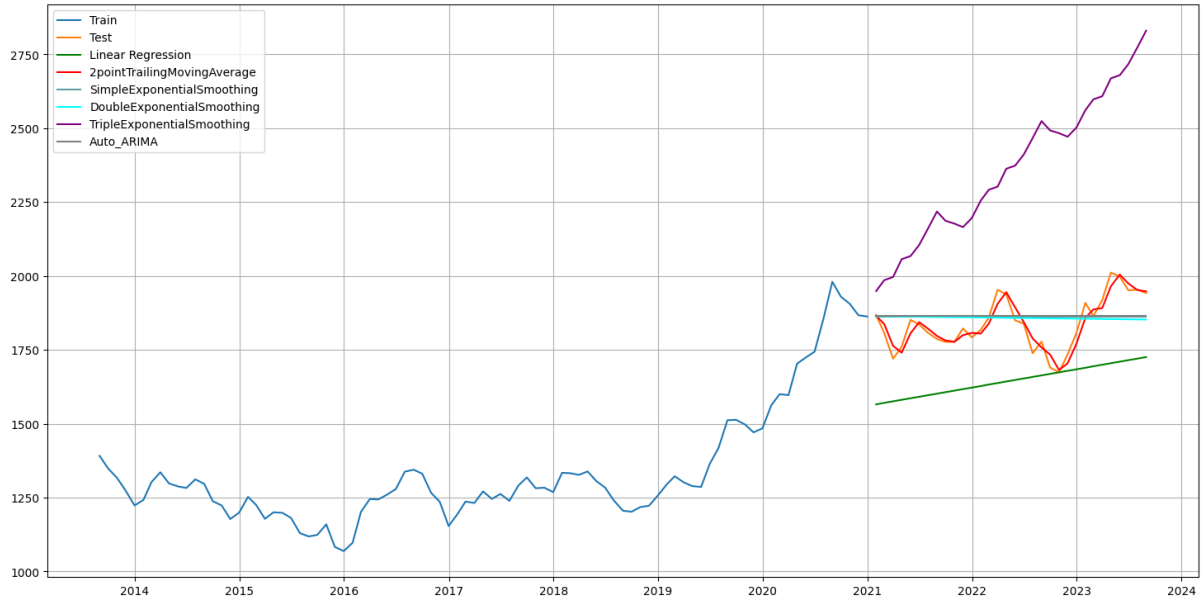

Figure 18 Plotting on both the Training and Test using autofit

7.2.7 Find the RMSE on test data

90.21680533104079

	RMSE
Auto_ARIMA	90.216805

7.3 ARIMA Model

- The following loop helps us in getting a combination of different parameters of p and q in the range of 0 and 2
- We have kept the value of d as 1 as we need to take a difference of the series to make it stationary.
- Define the range of AR values based on the ACF & PACF plots as below points.

7.3.1 Define the value to make the data stationary

Some parameter combinations for the Model...

```
Some parameter combinations for the Model...
Model: (0, 1, 1)
Model: (0, 1, 2)
Model: (1, 1, 0)
Model: (1, 1, 1)
Model: (1, 1, 2)
Model: (2, 1, 0)
Model: (2, 1, 1)
Model: (2, 1, 2)
```

Table 39 data stationary

7.3.2 Fit the ARIMA model

It is to find the AIC values by fit the ARIMA model. Use pd.concat to add new row to the dataframe.

```
ARIMA(0, 1, 0) - AIC:914.2597353789228
ARIMA(0, 1, 1) - AIC:908.1514792211835
ARIMA(0, 1, 2) - AIC:909.7965186418962
ARIMA(1, 1, 0) - AIC:908.4009658697732
ARIMA(1, 1, 1) - AIC:909.950670899301
ARIMA(1, 1, 2) - AIC:911.3728643896695
ARIMA(2, 1, 0) - AIC:909.6915224955326
ARIMA(2, 1, 1) - AIC:911.5729813396761
ARIMA(2, 1, 2) - AIC:912.9244622316618
```

Table 40 ARIMA model

7.3.3 Sort the above AIC values in the ascending order to get the parameters for the minimum AIC value

The minimum AIC value is 908.151479 so the respective param is (0, 1, 1).

	param	AIC
1	(0, 1, 1)	908.151479
10	(0, 1, 1)	908.151479
3	(1, 1, 0)	908.400966
12	(1, 1, 0)	908.400966
6	(2, 1, 0)	909.691522
15	(2, 1, 0)	909.691522
2	(0, 1, 2)	909.796519
11	(0, 1, 2)	909.796519
4	(1, 1, 1)	909.950671
13	(1, 1, 1)	909.950671
5	(1, 1, 2)	911.372864
14	(1, 1, 2)	911.372864
7	(2, 1, 1)	911.572981
16	(2, 1, 1)	911.572981
8	(2, 1, 2)	912.924462
17	(2, 1, 2)	912.924462
9	(0, 1, 0)	914.259735
0	(0, 1, 0)	914.259735

Table 41 parameters for the minimum AIC value

7.3.4 Build the ARIMA model based on the best params

Fit the ARIMA model with the summary result as appearing below.

```
                               SARIMAX Results
==============================================================================
Dep. Variable:                  Price   No. Observations:                   89
Model:                 ARIMA(0, 1, 1)   Log Likelihood                -452.076
Date:                Thu, 13 Feb 2025   AIC                            908.151
Time:                        16:25:49   BIC                            913.106
Sample:                    08-31-2013   HQIC                           910.148
                         - 12-31-2020
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ma.L1          0.2937      0.111      2.637      0.008       0.075       0.512
sigma2      1694.5828    243.026      6.973      0.000    1218.260    2170.905
===================================================================================
Ljung-Box (L1) (Q):                   0.01   Jarque-Bera (JB):                 2.45
Prob(Q):                              0.94   Prob(JB):                         0.29
Heteroskedasticity (H):               1.79   Skew:                             0.41
Prob(H) (two-sided):                  0.12   Kurtosis:                         3.09
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

Table 42 ARIMA model based on the best params

7.3.5 Forecast the ARIMA model

The predicted mean values were derived by forecast the ARIMA model.

	predicted_mean
2021-01-31	1864.874169
2021-02-28	1864.874169
2021-03-31	1864.874169
2021-04-30	1864.874169
2021-05-31	1864.874169
2021-06-30	1864.874169
2021-07-31	1864.874169
2021-08-31	1864.874169
2021-09-30	1864.874169
2021-10-31	1864.874169
2021-11-30	1864.874169
2021-12-31	1864.874169

2022-01-31 1864.874169

2022-02-28 1864.874169

2022-03-31 1864.874169

2022-04-30 1864.874169

2022-05-31 1864.874169

2022-06-30 1864.874169

2022-07-31 1864.874169

2022-08-31 1864.874169

2022-09-30 1864.874169

2022-10-31 1864.874169

2022-11-30 1864.874169

2022-12-31 1864.874169

2023-01-31 1864.874169

2023-02-28 1864.874169

2023-03-31 1864.874169

2023-04-30 1864.874169

2023-05-31 1864.874169

2023-06-30 1864.874169

2023-07-31 1864.874169

2023-08-31 1864.874169

dtype: float64

7.3.6 the RMSE on test data

90.21680533104079

7.3.7 Plotting on both the Training and Test using autofit

- This is for plotting the Training and Test using autofit to plot the predictions made by the linear regression model, best moving average model, simple exponential smoothening model, double exponential smoothening model, triple exponential smoothening model and ARIMA model.
- The legend box reflecting on the chart is defined the colour lines with respective names.

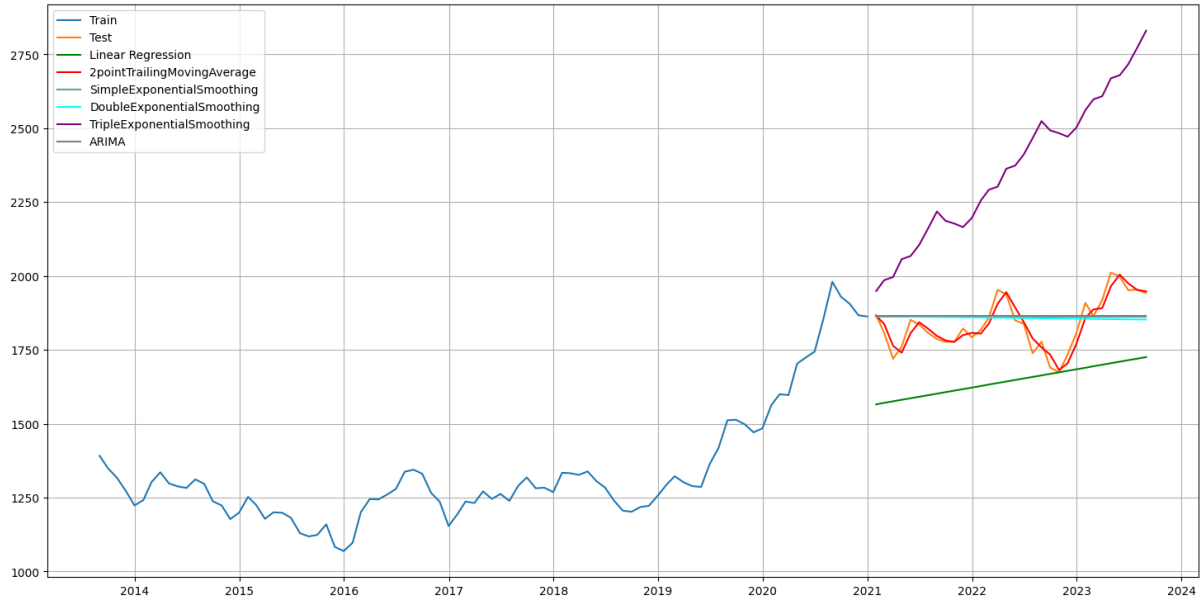

Figure 19 Plotting on both the Training and Test using autofit

	RMSE
Auto_ARIMA	90.216805
ARIMA	90.216805

Table 43 ARIMA model

Below are the different Trailing to Test RMSE.

	Test RMSE
Linear Regression	207.872126
2pointTrailingMovingAverage	27.944954
4pointTrailingMovingAverage	54.619271
6pointTrailingMovingAverage	70.894305
9pointTrailingMovingAverage	85.550202
Alpha=0.995,SimpleExponentialSmoothing	89.651792
Alpha=0.9,Beta=0.3,DoubleExponentialSmoothing	90.076443
Alpha=0.676,Beta=0.088,Gamma=0.323,TripleExponentialSmoothing	567.793388
Alpha=0.8,Beta=0.5,Gamma=0.5,TripleExponentialSmoothing	97.653133

Table 44 Test using autofit

7.4 Compare the Performance of the model & Forecast

The below comparison shows the Performance of the model & Forecast.

	RMSE
2pointTrailingMovingAverage	27.944954
4pointTrailingMovingAverage	54.619271
6pointTrailingMovingAverage	70.894305
9pointTrailingMovingAverage	85.550202
Alpha=0.995,SimpleExponentialSmoothing	89.651792
Alpha=0.9,Beta=0.3,DoubleExponentialSmoothing	90.076443
Auto_ARIMA	90.216805
ARIMA	90.216805
Alpha=0.8,Beta=0.5,Gamma=0.5,TripleExponentialSmoothing	97.653133
Linear Regression	207.872126
Alpha=0.676,Beta=0.088,Gamma=0.323,TripleExponentialSmoothing	567.793388

Table 45 Performance of the model & Forecast

7.4.1 Build the best model after comparing all the RMSE values

- Get the best model's name and its RMSE. Build and fit the best model (assuming 'Alpha=0.8, Beta=0.5, Gamma=0.5, TripleExponentialSmoothing' is the best model)
- Adjust this based on the results of your model comparison to Generate predictions using the best model. Plot the original data and the predictions of best model and to plot the best model.
- The legend displays the blue colour with original data, Test Data line by orange and green line with 2pointTrailingMovingAverage.

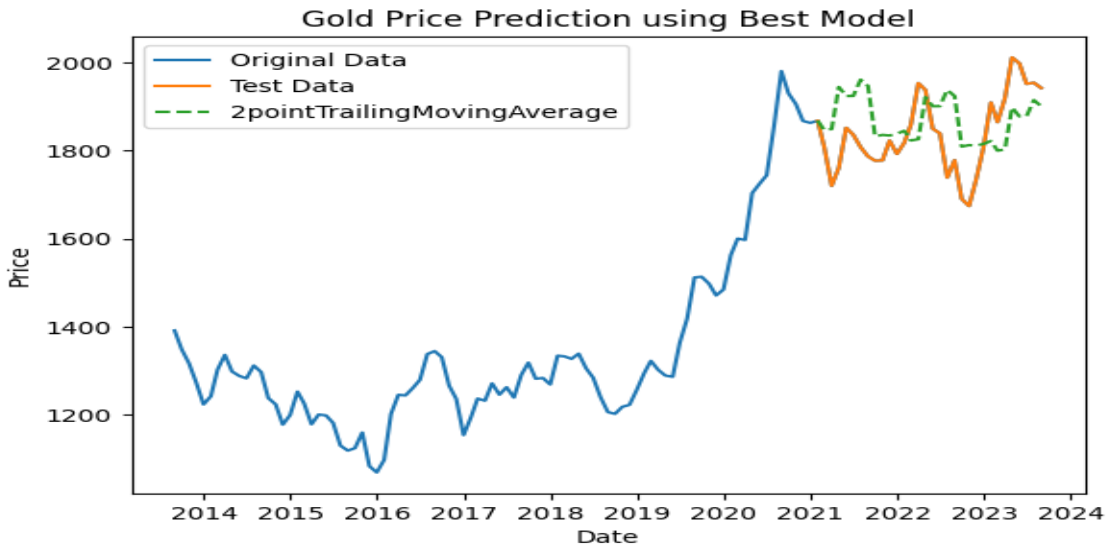

Figure 20 Build the best model after comparing all the RMSE values

7.4.2 to build the best model to forecast

- To fit the model. To forecast for the next 12 months. The Gold price forecasted is higher than the existing prices. The forecasted price is between the years 2021 and 2022 showing in orange line.
- The existing gold price is decreased from 2014 year to 2016 and then increased slightly up after that the price has gradually increased from 2019 to 2021. Later 2021 it has fluctuated and then raised to 2024.The existing prices are higher than the forecasted prices during 2021 and 2014 years.

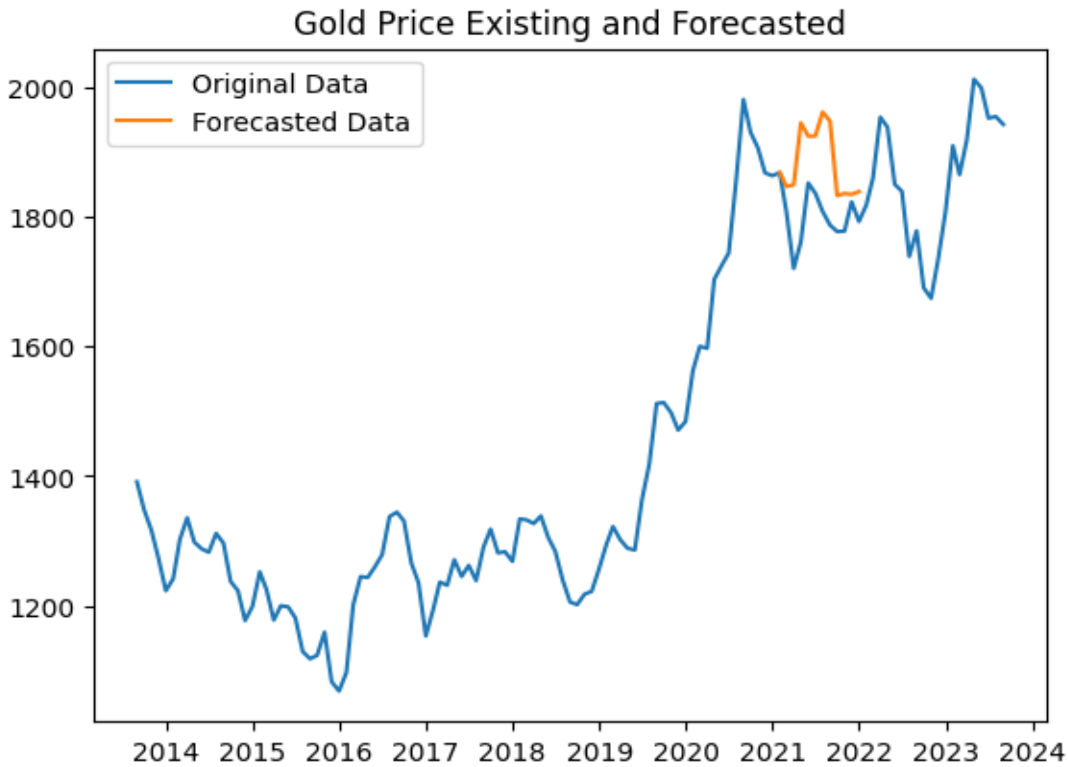

Figure 21 best model to forecasts

8. Actionable Insights and Recommendations

8.1 Insights

- We considered the missing values as the previous values so it is based on pure assumptions.
- Gold Prices has inherent volatility so very difficult to predictions.
- Monthly prices are very flexible so we can suggest to buy in December due to low average prices and recommend to sell in July or august months due to higher prices.
- The seasonal and trend chart shows prices clearly for the prediction even though difficulty.
- The raw data creates lot of confusions to apply the prediction models.
- Moving average and Exponential smoothing Models comparison gives multiple prices which leads to confuse the stakeholders.
- Forecast the best ARIMA best model based on the minimum AIC values.
- All the markets depend on these accurate predictions so it has too much risk and pressure involved.

8.2 Recommendations

- By adding the average values from the previous amount, it gives the better output closure to required results.
- The exploratory data analysis gives better visualization about gold prices in future to the investors, traders and stakeholders to give confidence.
- We can recommend the investors to use seasonal data for short term and trend is for long run for prediction analysis.
- It is better to prepare and transform the data in a form to perform ML models. So that the models are accurate and helpful to analyse the financial markets.
- Stakeholders may easily predict the financial market prices based on the minimum RMSE values represented by the best models.
- The minimum AIC values help to get the minimum param which can standardize the predication and minimize the variation of the prices.
- More accurate predictions help the companies for decision making and build the confidence to their clients by making best strategies to safeguard their investments.
- Business can expand based on this data analysis of these gold prices by increasing trust and bringing the investments.